Laboratory Manual

of

Data Visualization
BCA-609

M.M. INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT
MAHARISHI MARKANDESHWAR (DEEMED TO BE UNIVERSITY)
MULLANA–AMBALA, HARYANA (INDIA) - 133207
(Established under Section 3 of the UGC Act, 1956)
(Accredited by NAAC with Grade 'A++')

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Subject: Data Visualization Lab (Based on BCA-609) **BCA-609:** Data Visualization Lab	
LABORATORY MANUAL NO.: **BCA - 609**	ISSUE NO.: 01	ISSUE DATE: 08/01/26
LABORATORY: Data Visualization Lab	SEMESTER- VI	PAGE No.

MASTER LIST OF EXPERIMENTS

Sr. No.	EXPERIMENT No.	EXPERIMENT DESCRIPTION	Page No.
1.	BCA-609-01	Create a simple bar chart using student marks or sales data and explain what it shows.	
2.	BCA-609-02	Draw a line chart to represent time-based data (e.g., monthly attendance or sales).	
3.	BCA-609-03	Perform data representation vs data presentation for the same dataset using at least two different chart types.	
4.	BCA-609-04	Identify the purpose of data visualization for a given dataset and use a donut chart.	
5.	BCA-609-05	Explain the seven stages of data visualization using a simple example.	
6.	BCA-609-06	Create one basic visualization using any data visualization tool (Excel / Google Sheets / Tableau / Power BI).	
7.	BCA-609-07	Create an area chart for time-series data such as yearly profit or rainfall data.	
8.	BCA-609-08	Draw a scatter plot to show the relationship between two variables (e.g., study hours vs marks).	
9.	BCA-609-09	Create a pivot table and a pivot chart using a simple dataset.	
10.	BCA-609-10	Design a tree map to represent category-wise data (e.g., department-wise student count).	
11.	BCA-609-11	Draw a simple node-link diagram to show relationships (e.g., friends in a class).	
12.	BCA-609-12	Visualize the correlation between different numeric variables in a dataset using a color-coded matrix (heatmap).	
13.	BCA-609-13	Create a time-series visualization with multiple variables and analyze the observed patterns.	
14.	BCA-609-14	Perform text visualization by counting word frequency from a paragraph and plotting it.	
15.	BCA-609-15	Create a simple multivariate chart using more than two variables (e.g., marks in three subjects).	
16.	BCA-609-16	Study a small case example where visualization helps in understanding data easily.	
17.	BCA-609-17	Create a live-like data visualization using continuously updated values (manual update is allowed).	
18.	BCA-609-18	Write basic rules or best practices for creating good visualizations with examples.	

19.	BCA-609-19	Create a basic chart to show open and closed ports from given data.	
20	BCA-609-20	Create a firewall log visualization to identify suspicious traffic patterns.	
21.	BCA-609-21	Design an intrusion detection log visualization and highlight possible attacks.	
22.	BCA-609-22	Write a program to implement sentiment analysis on the live tweet data using Python libraries.	
23.	BCA-609-23	Write a program to create interactive visualizations to monitor device activity for Pre-processed simulated IoT data streams in real-time.	
24.	BCA-609-24	Design a secured data visualization system architecture and explain the security measures used.	

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a simple bar chart using student marks or sales data and explain what it shows.	

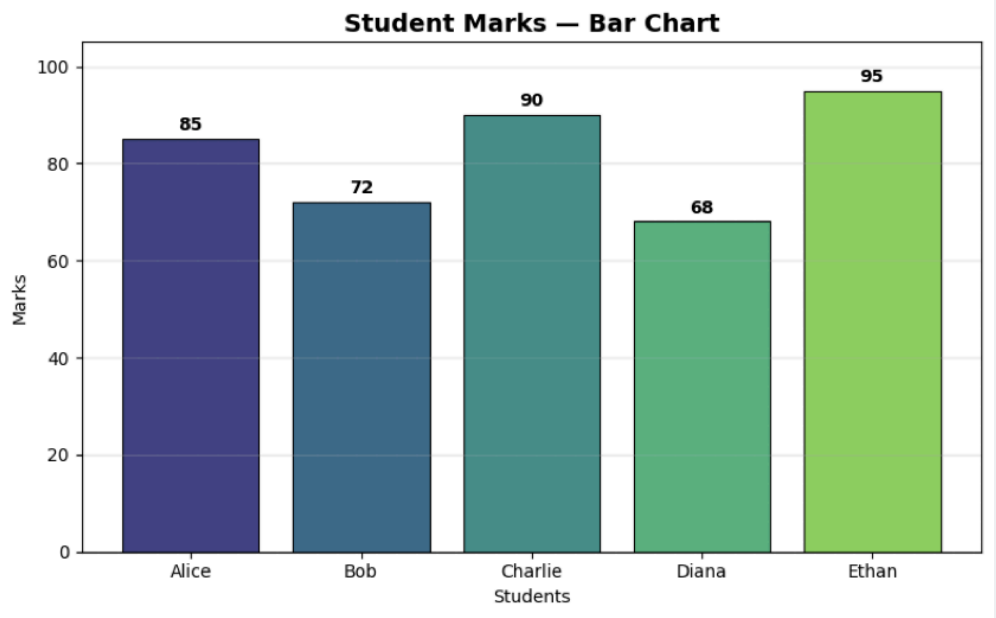
EXPERIMENT NO.: **BCA-609-01**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create a simple bar chart using student marks or sales data and explain what it shows.

```
import matplotlib.pyplot as plt
 import numpy as np

 # Student marks data
 students = ['Alice', 'Bob', 'Charlie', 'Diana', 'Ethan']
 marks = [85, 72, 90, 68, 95]

 colors = plt.cm.viridis(np.linspace(0.2, 0.8, len(students)))

 plt.figure(figsize=(8, 5))
 bars = plt.bar(students, marks, color=colors, edgecolor='black', linewidth=0.8)

 for bar, mark in zip(bars, marks):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
             str(mark), ha='center', va='bottom', fontweight='bold')

 plt.title('Student Marks — Bar Chart', fontsize=14, fontweight='bold')
 plt.xlabel('Students')
 plt.ylabel('Marks')
plt.ylim(0, 105)
 plt.grid(axis='y', alpha=0.3)
 plt.tight_layout()
 plt.show()

 print("\n--- Explanation ---")
 print("The bar chart shows marks of 5 students.")
 print("Each bar represents a student; the height shows their marks.")
 print("Ethan scored highest (95) while Diana scored lowest (68).")
```

Matplotlib is building the font cache; this may take a moment.

OutPut:



Explanation:

The bar chart shows the marks of 5 students.
Each bar represents a student; the height shows their marks.
Ethan scored highest (95) while Diana scored lowest (68).

Prepared By: Ms. Chetna Thakur	**Approved By:**

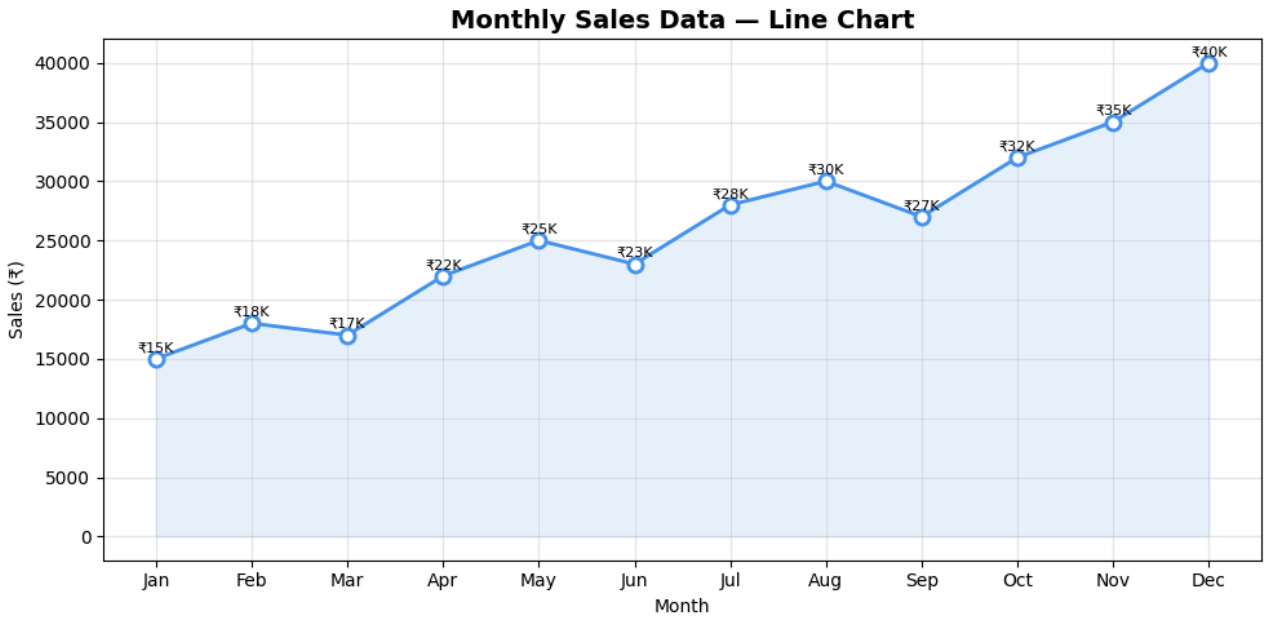
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Draw a line chart to represent time-based data (e.g., monthly attendance or sales).	

EXPERIMENT NO.: **BCA-609-02**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Draw a line chart to represent time-based data (e.g., monthly attendance or sales).

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
sales = [15000, 18000, 17000, 22000, 25000, 23000,
         28000, 30000, 27000, 32000, 35000, 40000]

plt.figure(figsize=(10, 5))
plt.plot(months, sales, marker='o', color='#2196F3', linewidth=2,
         markersize=8, markerfacecolor='white', markeredgewidth=2)

for i, val in enumerate(sales):
    plt.text(i, val + 600, f'₹{val//1000}K', ha='center', fontsize=8)

plt.title('Monthly Sales Data — Line Chart', fontsize=14, fontweight='bold')
plt.xlabel('Month')
plt.ylabel('Sales (₹)')
plt.grid(True, alpha=0.3)
plt.fill_between(range(len(months)), sales, alpha=0.1, color='#2196F3')
plt.tight_layout()
plt.show()

print("\n--- Explanation ---")
print("The line chart shows a general upward trend in sales throughout the year.")
print("Sales grew from ₹15,000 in January to ₹40,000 in December.")
```

OutPut:



Explanation

The line chart shows a general upward trend in sales throughout the year.
Sales grew from ₹15,000 in January to ₹40,000 in December.

Prepared By: Ms. Chetna Thakur	**Approved By:**

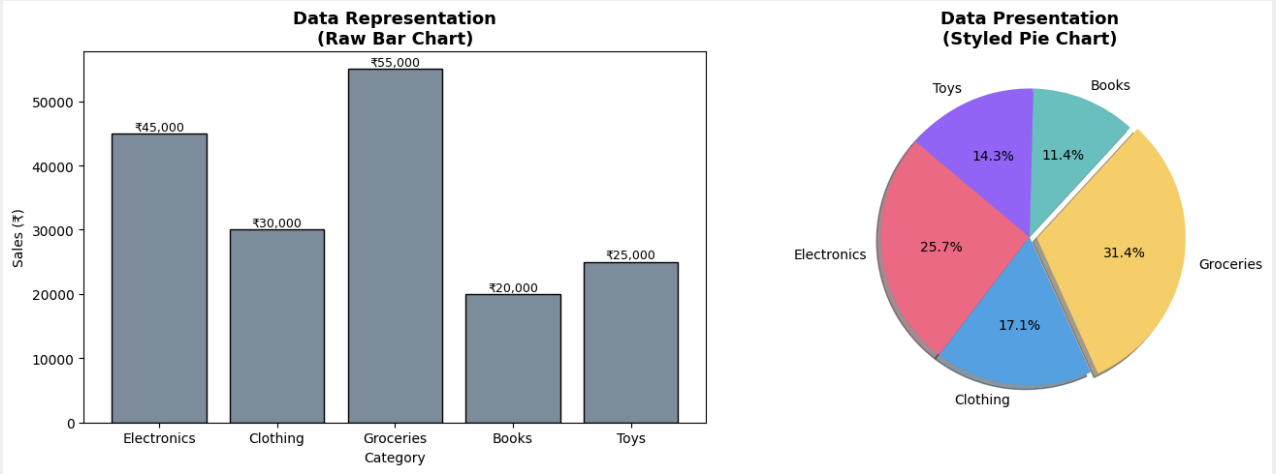
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Perform data representation vs data presentation for the same dataset using at least two different chart types.	

EXPERIMENT NO.: **BCA-609-03**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Perform data representation vs data presentation for the same dataset using at least two different chart types.

```
import matplotlib.pyplot as plt
 import numpy as np

 categories = ['Electronics', 'Clothing', 'Groceries', 'Books', 'Toys']
 sales = [45000, 30000, 55000, 20000, 25000]

 fig, axes = plt.subplots(1, 2, figsize=(14, 5))

 # Data Representation — Raw bar chart
 bars = axes[0].bar(categories, sales, color='#78909C', edgecolor='black')
 axes[0].set_title('Data Representation\n(Raw Bar Chart)', fontsize=13, fontweight='bold')
 axes[0].set_ylabel('Sales (₹)')
 axes[0].set_xlabel('Category')
 for bar, s in zip(bars, sales):
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+500,
                 f'₹{s:,}', ha='center', fontsize=9)

 # Data Presentation — Styled pie chart
 colors = ['#FF6384','#36A2EB','#FFCE56','#4BC0C0','#9966FF']
 explode = (0, 0, 0.05, 0, 0)
 axes[1].pie(sales, labels=categories, autopct='%1.1f%%', colors=colors,
             explode=explode, shadow=True, startangle=140)
 axes[1].set_title('Data Presentation\n(Styled Pie Chart)', fontsize=13, fontweight='bold')

 plt.tight_layout()
 plt.show()

 print("\n--- Explanation ---")
 print("Data Representation: Shows raw numbers using a bar chart — focuses on exact values.")
 print("Data Presentation : Shows proportions using a pie chart — focuses on storytelling & insight.")
 print("Groceries lead sales (31.4%), followed by Electronics (25.7%).")
```

OutPut:



Explanation

Data Representation: Shows raw numbers using a bar chart — focuses on exact values.
Data Presentation : Shows proportions using a pie chart — focuses on storytelling & insight.
Groceries lead sales (31.4%), followed by Electronics (25.7%).

Prepared By: Ms. Chetna Thakur	**Approved By:**

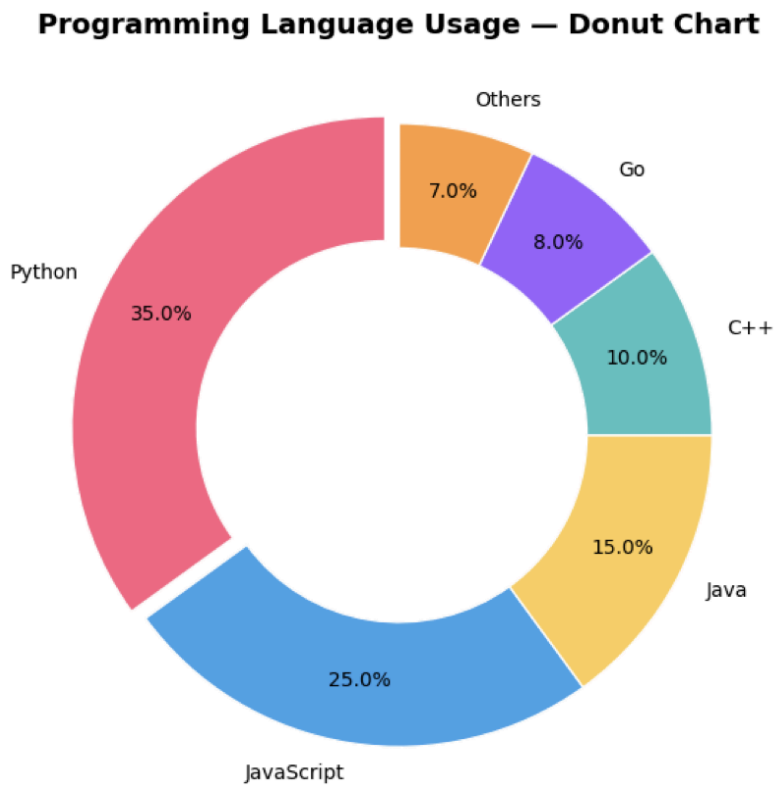
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Identify the purpose of data visualization for a given dataset and use a donut chart.	

EXPERIMENT NO.: **BCA-609-04**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Identify the purpose of data visualization for a given dataset and use a donut chart.

```
import matplotlib.pyplot as plt

labels = ['Python', 'JavaScript', 'Java', 'C++', 'Go', 'Others']
usage = [35, 25, 15, 10, 8, 7]
colors = ['#FF6384','#36A2EB','#FFCE56','#4BC0C0','#9966FF','#FF9F40']
explode = (0.05, 0, 0, 0, 0, 0)

fig, ax = plt.subplots(figsize=(8, 6))
wedges, texts, autotexts = ax.pie(usage, labels=labels, autopct='%1.1f%%',
    colors=colors, explode=explode, startangle=90,
    pctdistance=0.8, wedgeprops=dict(width=0.4, edgecolor='white'))

centre_circle = plt.Circle((0, 0), 0.55, fc='white')
ax.add_artist(centre_circle)
ax.set_title('Programming Language Usage — Donut Chart', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n--- Purpose of Data Visualization ---")
print("1. Simplifies complex data into understandable visual forms")
print("2. Reveals patterns, trends, and outliers quickly")
print("3. Supports better decision-making")
print("4. Communicates insights effectively to both technical and non-technical audiences")
print("\nThe donut chart clearly shows Python dominates with 35% usage.")
```

OutPut:



Purpose of Data Visualization

1. Simplifies complex data into understandable visual forms
2. Reveals patterns, trends, and outliers quickly
3. Supports better decision-making
4. Communicates insights effectively to both technical and non-technical audiences

The donut chart clearly shows Python dominates with 35% usage.

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Explain the seven stages of data visualization using a simple example.	

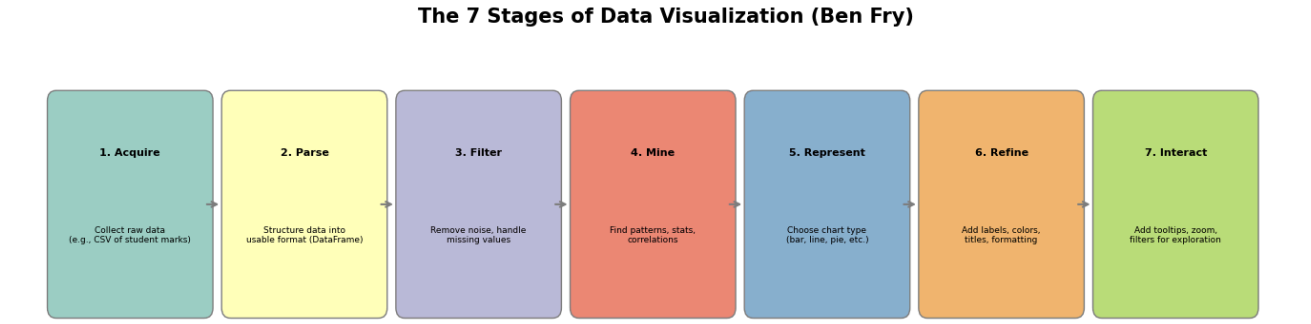
EXPERIMENT NO.: **BCA-609-05**	ISSUE No. : 00	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Explain the seven stages of data visualization using a simple example.

```
import matplotlib.pyplot as plt
 import matplotlib.patches as mpatches

 stages = [
    ("1. Acquire", "Collect raw data\n(e.g., CSV of student marks)"),
    ("2. Parse", "Structure data into\nusable format (DataFrame)"),
    ("3. Filter", "Remove noise, handle\nmissing values"),
    ("4. Mine", "Find patterns, stats,\ncorrelations"),
    ("5. Represent", "Choose chart type\n(bar, line, pie, etc.)"),
    ("6. Refine", "Add labels, colors,\ntitles, formatting"),
    ("7. Interact", "Add tooltips, zoom,\nfilters for exploration"),
 ]

 fig, ax = plt.subplots(figsize=(14, 4))
 ax.set_xlim(0, 15)
 ax.set_ylim(0, 3)
 ax.axis('off')
 ax.set_title('The 7 Stages of Data Visualization (Ben Fry)', fontsize=15, fontweight='bold', pad=20)

 colors = plt.cm.Set3(range(len(stages)))
 for i, (title, desc) in enumerate(stages):
    x = i * 2 + 0.5
    rect = mpatches.FancyBboxPatch((x, 0.5), 1.7, 2, boxstyle="round,pad=0.1",
                                   facecolor=colors[i], edgecolor='gray')
    ax.add_patch(rect)
    ax.text(x + 0.85, 2.0, title, ha='center', va='center', fontweight='bold', fontsize=8)
    ax.text(x + 0.85, 1.2, desc, ha='center', va='center', fontsize=6.5)
    if i < len(stages) - 1:
       ax.annotate('', xy=(x + 1.9, 1.5), xytext=(x + 1.7, 1.5),
                   arrowprops=dict(arrowstyle='->', color='gray', lw=1.5))

 plt.tight_layout()
 plt.show()

 print("\n--- Example: Visualizing Student Exam Results ---")
 print("Stage 1 — Acquire : Collect marks from exam sheets into a CSV file")
 print("Stage 2 — Parse   : Load CSV into a Pandas DataFrame")
 print("Stage 3 — Filter  : Remove students with incomplete records")
```

```
print("Stage 4 — Mine    : Calculate averages, find toppers, identify failing students")
print("Stage 5 — Represent: Choose a bar chart to compare marks")
print("Stage 6 — Refine  : Add colors, labels, title, and grid lines")
print("Stage 7 — Interact: Allow filtering by subject or sorting by marks")


--- Example: Visualizing Student Exam Results ---
Stage 1 — Acquire : Collect marks from exam sheets into a CSV file
Stage 2 — Parse   : Load CSV into a Pandas DataFrame
Stage 3 — Filter  : Remove students with incomplete records
Stage 4 — Mine    : Calculate averages, find toppers, identify failing students
Stage 5 — Represent: Choose a bar chart to compare marks
Stage 6 — Refine  : Add colors, labels, title, and grid lines
Stage 7 — Interact: Allow filtering by subject or sorting by marks
```

OutPut:



Example: Visualizing Student Exam Results
Stage 1 — Acquire : Collect marks from exam sheets into a CSV file
Stage 2 — Parse   : Load CSV into a Pandas DataFrame
Stage 3 — Filter : Remove students with incomplete records
Stage 4 — Mine   : Calculate averages, find toppers, identify failing students
Stage 5 — Represent: Choose a bar chart to compare marks
Stage 6 — Refine : Add colors, labels, title, and grid lines
Stage 7 — Interact: Allow filtering by subject or sorting by marks

Prepared By: Ms. Chetna Thakur	Approved By:

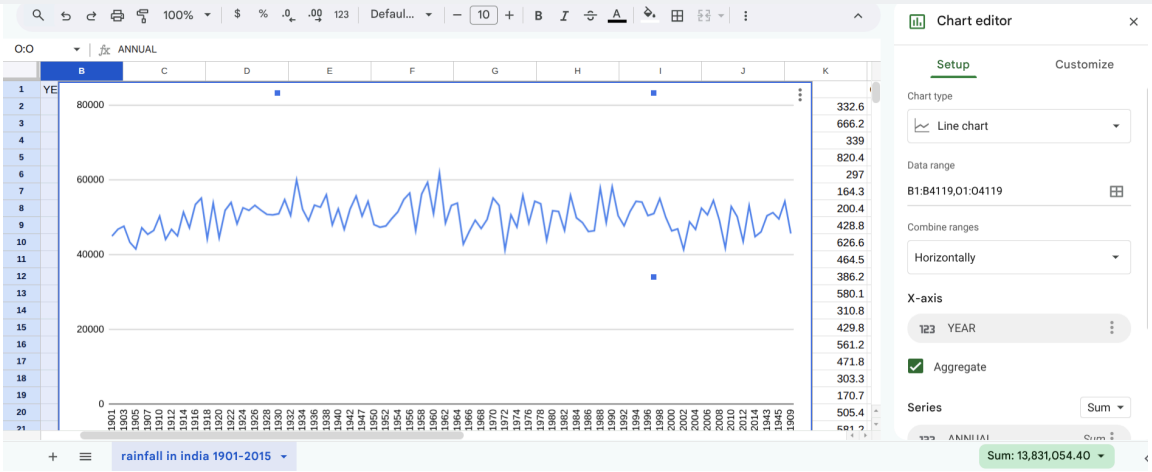
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create one basic visualization using any data visualization tool (Excel / Google Sheets / Tableau / Power BI).	

EXPERIMENT NO.: **BCA-609-06**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create one basic visualization using any data visualization tool (Excel / Google Sheets / Tableau / Power BI).

```
from IPython.display import Image

    image_path = "data/practicle6.png"
    Image(filename=image_path)
```

OutPut:



Prepared By: Ms. Chetna Thakur	Approved By:

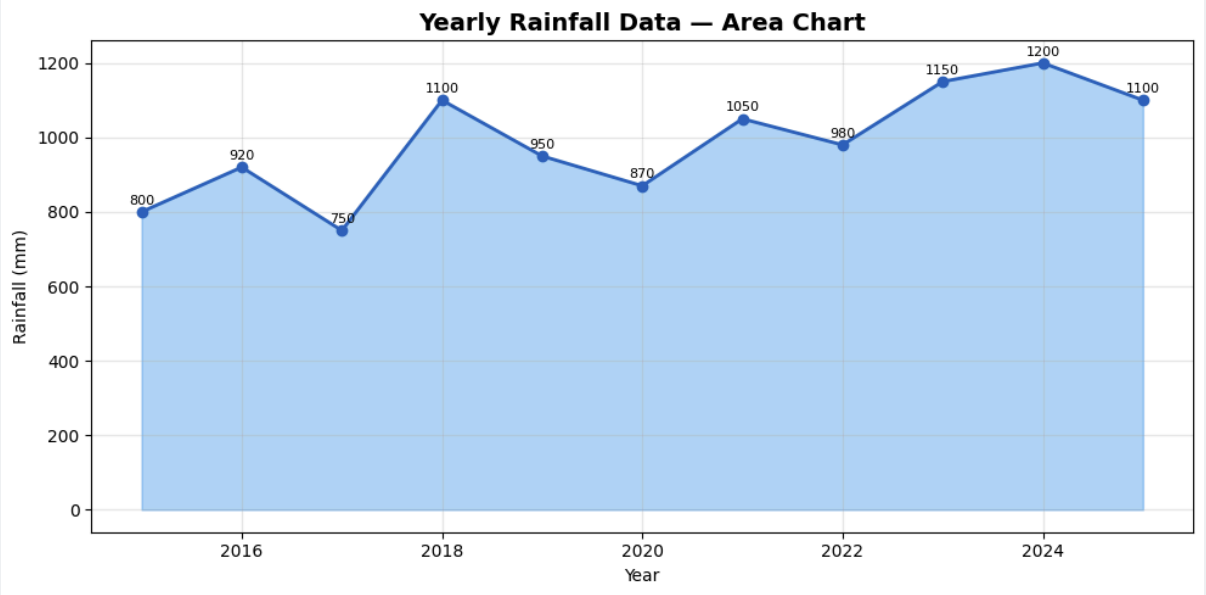
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create an area chart for time-series data such as yearly profit or rainfall data.	

EXPERIMENT NO.: **BCA-609-07**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create an area chart for time-series data such as yearly profit or rainfall data.

```
import matplotlib.pyplot as plt
 import numpy as np

 years = list(range(2015, 2026))
 rainfall = [800, 920, 750, 1100, 950, 870, 1050, 980, 1150, 1200, 1100]

 plt.figure(figsize=(10, 5))
 plt.fill_between(years, rainfall, alpha=0.4, color='#2196F3')
 plt.plot(years, rainfall, marker='o', color='#1565C0', linewidth=2)

 for x, y in zip(years, rainfall):
    plt.text(x, y + 20, str(y), ha='center', fontsize=8)

 plt.title('Yearly Rainfall Data — Area Chart', fontsize=14, fontweight='bold')
 plt.xlabel('Year')
 plt.ylabel('Rainfall (mm)')
 plt.grid(True, alpha=0.3)
 plt.tight_layout()
 plt.show()

 print("\n--- Explanation ---")
 print("The area chart visualizes rainfall over 11 years (2015–2025).")
 print("The shaded area emphasizes the cumulative volume of rainfall.")
 print("Peak rainfall was in 2024 (1200mm); lowest was in 2017 (750mm).")
```

OutPut:



Explanation

The area chart visualizes rainfall over 11 years (2015–2025).
The shaded area emphasizes the cumulative volume of rainfall.
Peak rainfall was in 2024 (1200mm); lowest was in 2017 (750mm).

Prepared By: Ms. Chetna Thakur	**Approved By:**

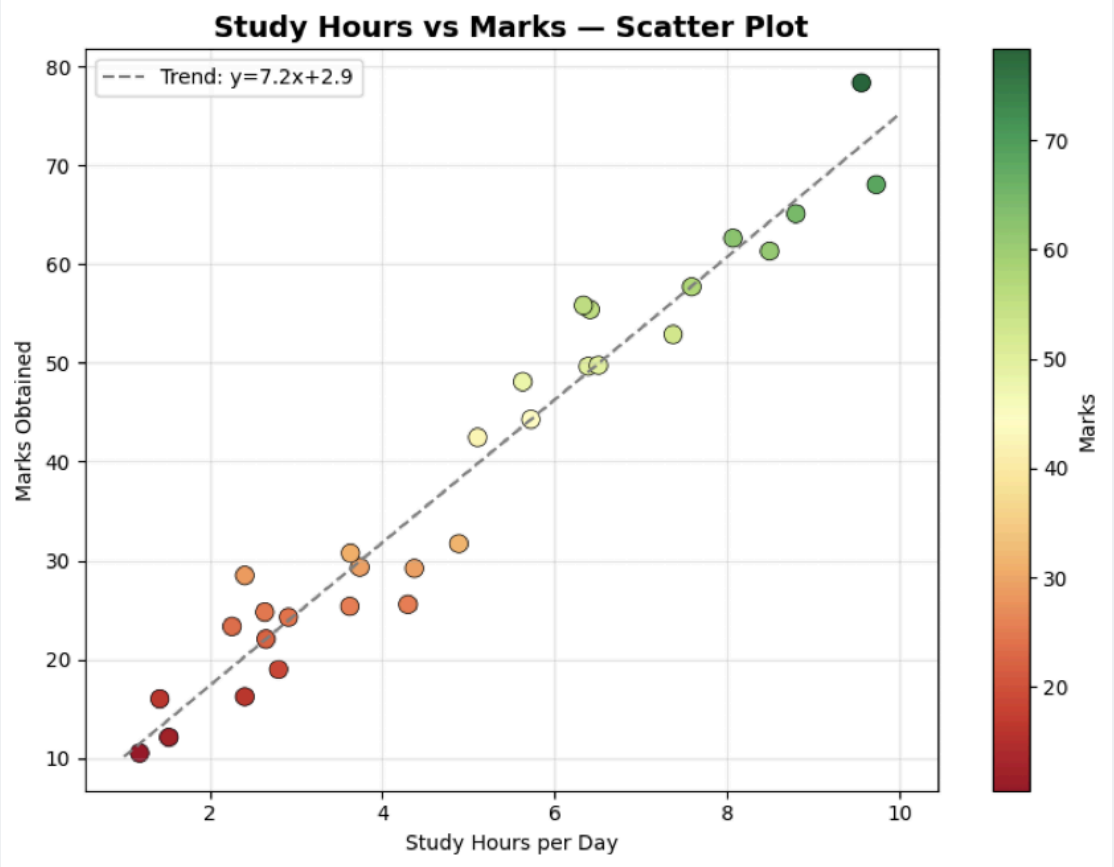
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Draw a scatter plot to show the relationship between two variables (e.g., study hours vs marks).	

EXPERIMENT NO.: **BCA-609-08**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Draw a scatter plot to show the relationship between two variables (e.g., study hours vs marks).

```
import matplotlib.pyplot as plt
 import numpy as np

 np.random.seed(42)
 study_hours = np.random.uniform(1, 10, 30)
 marks = 8 * study_hours + np.random.normal(0, 5, 30)
 marks = np.clip(marks, 0, 100)

 plt.figure(figsize=(8, 6))
 plt.scatter(study_hours, marks, c=marks, cmap='RdYlGn', s=80, edgecolor='black', linewidth=0.5)

 # Trend line
 z = np.polyfit(study_hours, marks, 1)
 p = np.poly1d(z)
 x_line = np.linspace(1, 10, 100)
 plt.plot(x_line, p(x_line), '--', color='gray', linewidth=1.5, label=f'Trend: y={z[0]:.1f}x+{z[1]:.1f}')

 plt.colorbar(label='Marks')
 plt.title('Study Hours vs Marks — Scatter Plot', fontsize=14, fontweight='bold')
 plt.xlabel('Study Hours per Day')
 plt.ylabel('Marks Obtained')
 plt.legend()
 plt.grid(True, alpha=0.3)
 plt.tight_layout()
 plt.show()

 print("\n--- Explanation ---")
 print("The scatter plot shows a positive correlation between study hours and marks.")
 print(f"Trend line equation: Marks ≈ {z[0]:.1f} × Hours + {z[1]:.1f}")
 print("Students studying more hours tend to score higher marks.")
```

OutPut:



Explanation

The scatter plot shows a positive correlation between study hours and marks.
Trend line equation: Marks ≈ 7.2 × Hours + 2.9
Students studying more hours tend to score higher marks.

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a pivot table and a pivot chart using a simple dataset.	

EXPERIMENT NO.: **BCA-609-09**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create a pivot table and a pivot chart using a simple dataset.

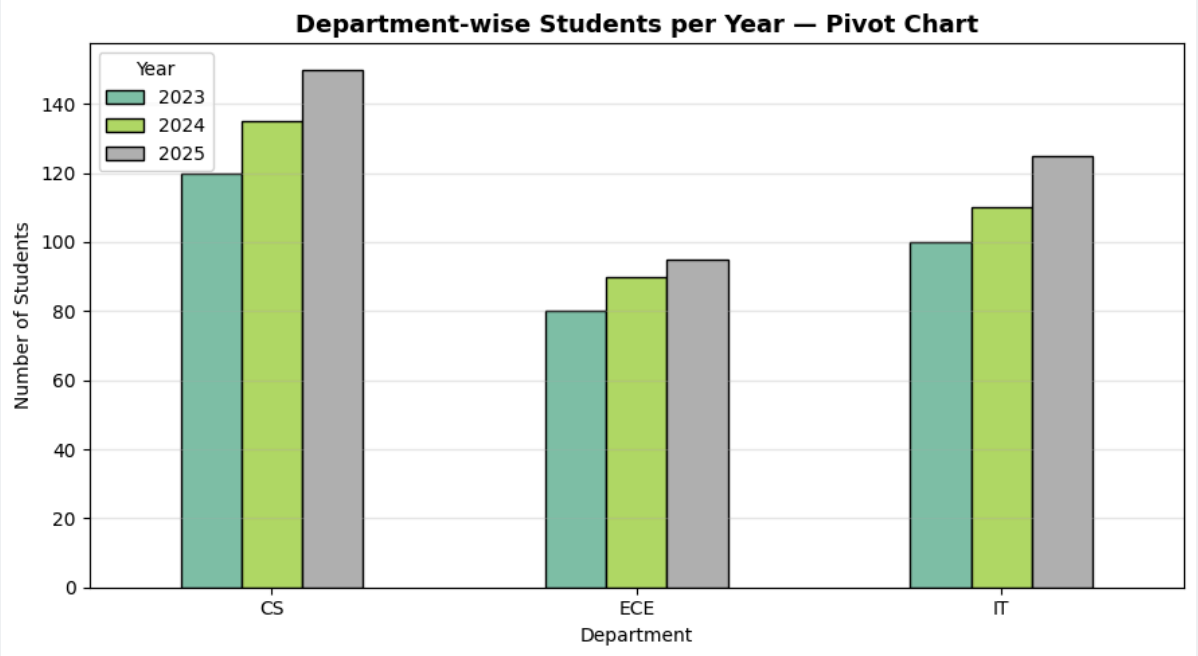
```
import pandas as pd
 import matplotlib.pyplot as plt

 data = pd.DataFrame({
    'Department': ['CS','CS','CS','IT','IT','IT','ECE','ECE','ECE'],
    'Year': ['2023','2024','2025','2023','2024','2025','2023','2024','2025'],
    'Students': [120, 135, 150, 100, 110, 125, 80, 90, 95]
 })
 print("--- Original Data ---")
 print(data.to_string(index=False))

 pivot = data.pivot_table(values='Students', index='Department', columns='Year', aggfunc='sum')
 print("\n--- Pivot Table ---")
 print(pivot)

 pivot.plot(kind='bar', figsize=(9, 5), colormap='Set2', edgecolor='black')
 plt.title('Department-wise Students per Year — Pivot Chart', fontsize=13, fontweight='bold')
 plt.ylabel('Number of Students')
 plt.xticks(rotation=0)
 plt.legend(title='Year')
 plt.grid(axis='y', alpha=0.3)
 plt.tight_layout()
 plt.show()

 print("\nInsight: CS department consistently has the highest enrollment.")
```

```
--- Original Data ---
 Department Year  Students
         CS 2023       120
         CS 2024       135
         CS 2025       150
         IT 2023       100
         IT 2024       110
         IT 2025       125
        ECE 2023        80
        ECE 2024        90
        ECE 2025        95
```

--- Pivot Table ---

Year	2023	2024	2025
Department			
CS	120	135	150
ECE	80	90	95
IT	100	110	125

OutPut:



Explanation

Insight: CS department consistently has the highest enrollment.

Prepared By: Ms. Chetna Thakur	Approved By:

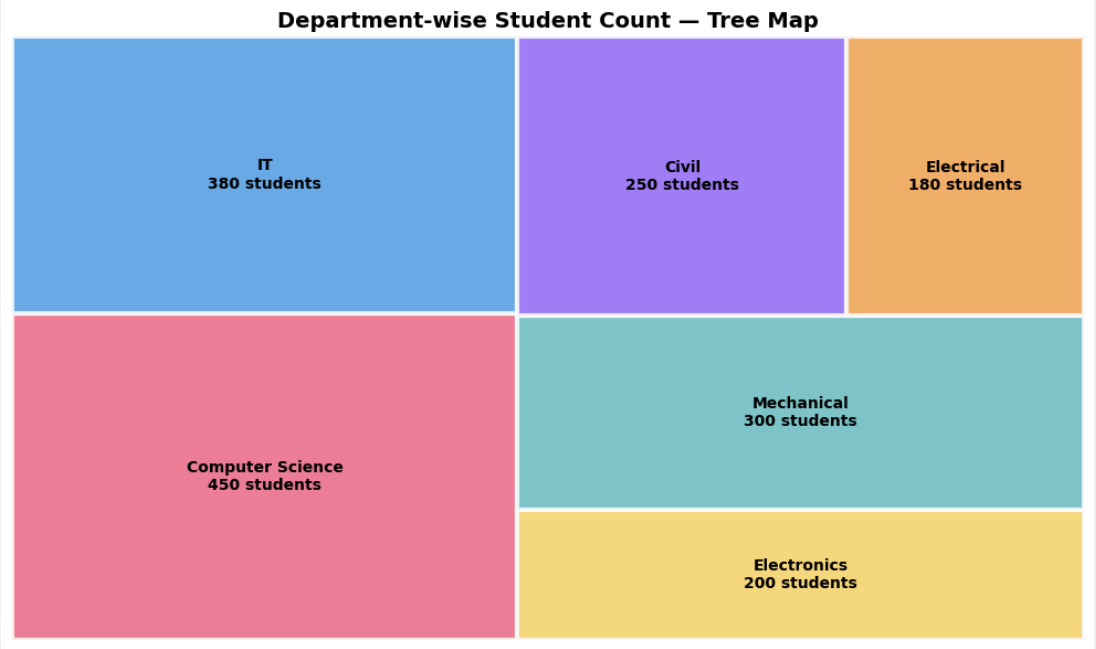
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Design a tree map to represent category-wise data (e.g., department-wise student count).	

EXPERIMENT NO.: **BCA-609-10**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Design a tree map to represent category-wise data (e.g., department-wise student count).

```
try:
    import squarify
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'squarify', '-q'])
    import squarify
import matplotlib.pyplot as plt

departments = ['Computer Science', 'IT', 'Electronics', 'Mechanical', 'Civil', 'Electrical']
students = [450, 380, 200, 300, 250, 180]
colors = ['#FF6384','#36A2EB','#FFCE56','#4BC0C0','#9966FF','#FF9F40']

labels = [f'{d}\n{s} students' for d, s in zip(departments, students)]

plt.figure(figsize=(10, 6))
squarify.plot(sizes=students, label=labels, color=colors, alpha=0.85,
              edgecolor='white', linewidth=3, text_kwargs={'fontsize': 10, 'fontweight': 'bold'})
plt.title('Department-wise Student Count — Tree Map', fontsize=14, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()

print("\n--- Explanation ---")
print("The tree map shows relative sizes of departments by student count.")
print("Larger rectangles = more students. CS is the largest department (450).")
```

OutPut:



Explanation

The tree map shows relative sizes of departments by student count.
Larger rectangles = more students. CS is the largest department (450).

Prepared By: Ms. Chetna Thakur	**Approved By:**

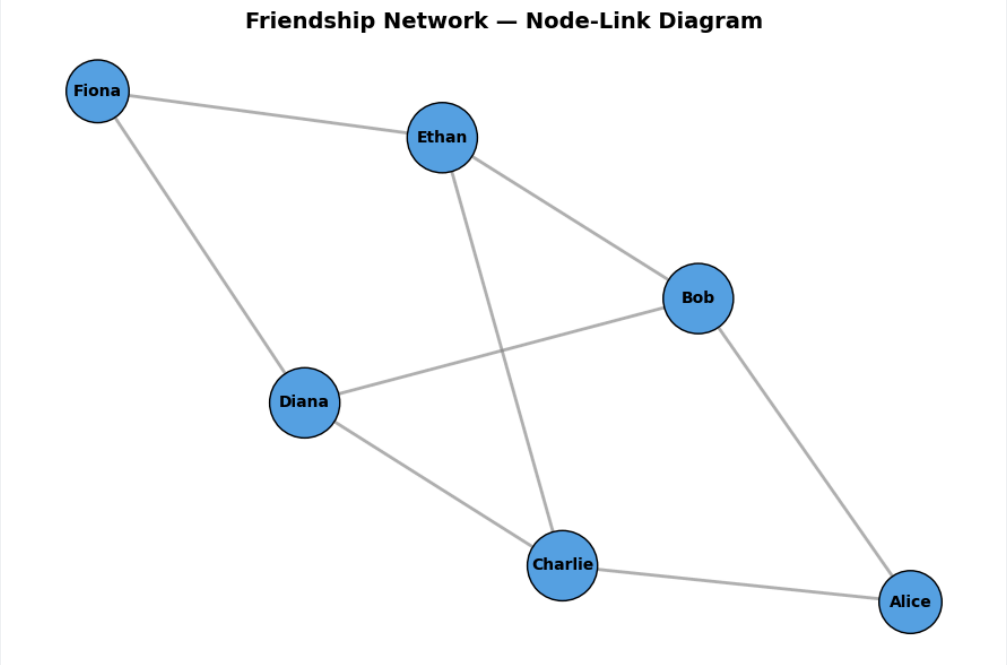
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Draw a simple node-link diagram to show relationships (e.g., friends in a class).	

EXPERIMENT NO.: **BCA-609-11**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Draw a simple node-link diagram to show relationships (e.g., friends in a class). CO2 / PO5

```
try:
    import networkx as nx
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'networkx', '-q'])
    import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
friendships = [
    ('Alice', 'Bob'), ('Alice', 'Charlie'), ('Bob', 'Diana'),
    ('Charlie', 'Diana'), ('Charlie', 'Ethan'), ('Diana', 'Fiona'),
    ('Ethan', 'Fiona'), ('Bob', 'Ethan')
]
G.add_edges_from(friendships)

plt.figure(figsize=(9, 6))
pos = nx.spring_layout(G, seed=42)
node_sizes = [800 + 400 * G.degree(n) for n in G.nodes()]
nx.draw_networkx_nodes(G, pos, node_size=node_sizes, node_color='#36A2EB', edgecolors='black')
nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
nx.draw_networkx_edges(G, pos, width=2, alpha=0.6, edge_color='gray')

plt.title('Friendship Network — Node-Link Diagram', fontsize=14, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()

print("\n--- Explanation ---")
print("Each node represents a student; edges represent friendships.")
print("Larger nodes have more connections (higher degree).")
for n in G.nodes():
    print(f"  {n}: {G.degree(n)} friends")
```

OutPut:



Explanation

Each node represents a student; edges represent friendships.
Larger nodes have more connections (higher degree).

Alice: 2 friends
Bob: 3 friends
Charlie: 3 friends
Diana: 3 friends
Ethan: 3 friends
Fiona: 2 friends

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Visualize the correlation between different numeric variables using a color-coded matrix (heatmap).	

EXPERIMENT NO.: **BCA-609-12**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

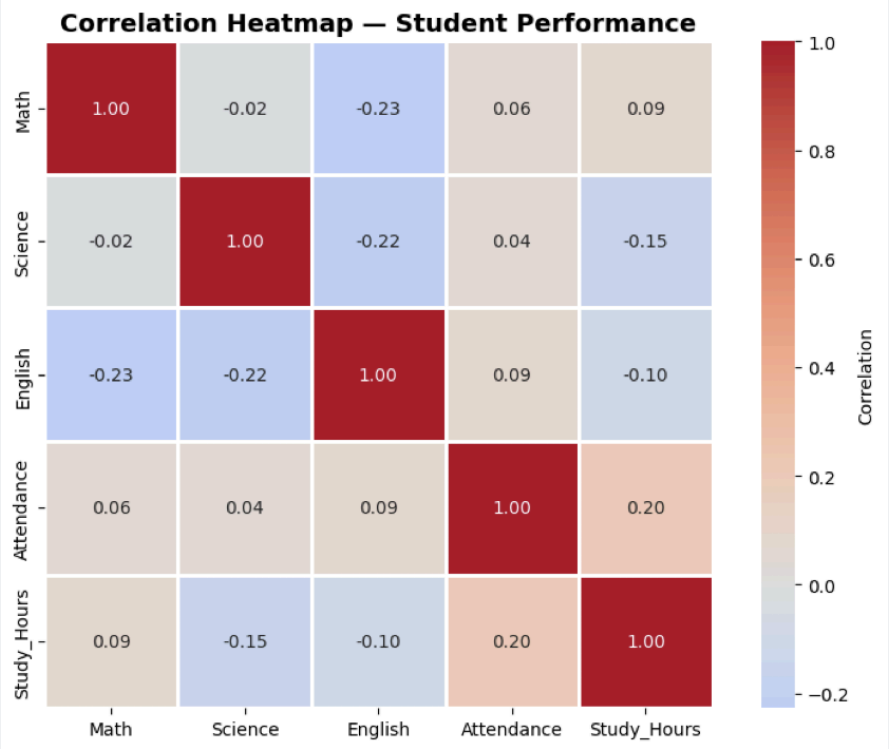
Objective: Visualize the correlation between different numeric variables using a color-coded matrix (heatmap).

```
import pandas as pd
 import numpy as np
 import matplotlib.pyplot as plt
 try:
    import seaborn as sns
 except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'seaborn', '-q'])
    import seaborn as sns

 np.random.seed(42)
 n = 50
 data = pd.DataFrame({
    'Math': np.random.randint(40, 100, n),
    'Science': np.random.randint(40, 100, n),
    'English': np.random.randint(40, 100, n),
    'Attendance': np.random.randint(50, 100, n),
    'Study_Hours': np.random.uniform(1, 8, n).round(1)
 })

 corr = data.corr()
 print("--- Correlation Matrix ---")
 print(corr.round(2))

 plt.figure(figsize=(8, 6))
 sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, linewidths=1,
             fmt='.2f', square=True, cbar_kws={'label': 'Correlation'})
 plt.title('Correlation Heatmap — Student Performance', fontsize=14, fontweight='bold')
 plt.tight_layout()
 plt.show()

 print("\n--- Explanation ---")
 print("The heatmap visualizes correlations between numeric variables.")
 print("Red = positive correlation, Blue = negative correlation.")
 print("Values close to 1 or -1 indicate strong relationships.")
```

--- Correlation Matrix ---

	Math	Science	English	Attendance	Study_Hours
Math	1.00	-0.02	-0.23	0.06	0.09
Science	-0.02	1.00	-0.22	0.04	-0.15
English	-0.23	-0.22	1.00	0.09	-0.10
Attendance	0.06	0.04	0.09	1.00	0.20
Study_Hours	0.09	-0.15	-0.10	0.20	1.00

OutPut:



Explanation

The heatmap visualizes correlations between numeric variables.
Red = positive correlation, Blue = negative correlation.
Values close to 1 or -1 indicate strong relationships.

Prepared By: Ms. Chetna Thakur	Approved By:

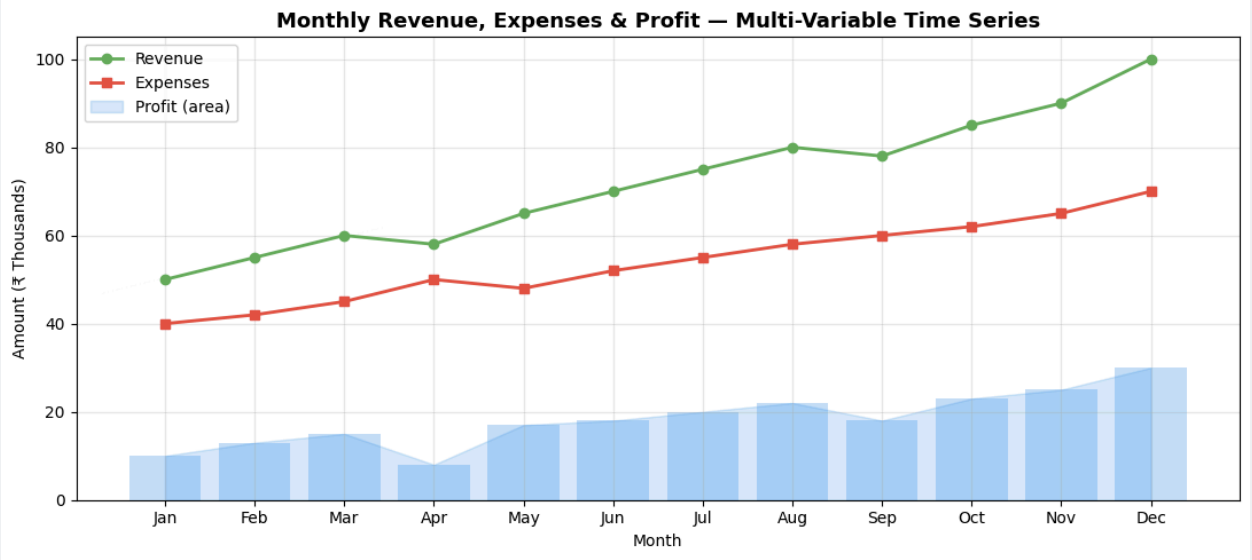
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a time-series visualization with multiple variables and analyze the observed patterns.	

EXPERIMENT NO.: **BCA-609-13**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create a time-series visualization with multiple variables and analyze the observed patterns.

```
import matplotlib.pyplot as plt
 import numpy as np

 months = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
 revenue = [50, 55, 60, 58, 65, 70, 75, 80, 78, 85, 90, 100]
 expenses = [40, 42, 45, 50, 48, 52, 55, 58, 60, 62, 65, 70]
 profit = [r - e for r, e in zip(revenue, expenses)]

 fig, ax1 = plt.subplots(figsize=(11, 5))
 ax1.plot(months, revenue, 'o-', color='#4CAF50', linewidth=2, label='Revenue')
 ax1.plot(months, expenses, 's-', color='#F44336', linewidth=2, label='Expenses')
 ax1.fill_between(months, profit, alpha=0.2, color='#2196F3', label='Profit (area)')
 ax1.bar(months, profit, alpha=0.3, color='#2196F3')

 ax1.set_xlabel('Month')
 ax1.set_ylabel('Amount (₹ Thousands)')
   ax1.set_title('Monthly Revenue, Expenses & Profit — Multi-Variable Time Series', fontsize=13,
fontweight='bold')
 ax1.legend(loc='upper left')
 ax1.grid(True, alpha=0.3)
 plt.tight_layout()
 plt.show()

 print("\n--- Analysis ---")
 print("1. Revenue shows steady growth from ₹50K to ₹100K (100% increase)")
 print("2. Expenses also grew but at a slower rate (₹40K to ₹70K = 75%)")
 print("3. Profit margin improved from ₹10K to ₹30K — a positive trend")
 print("4. Slight dip in April revenue and September suggest seasonal effects")
```

OutPut:



Explanation

Analysis ---

1. Revenue shows steady growth from ₹50K to ₹100K (100% increase)
2. Expenses also grew but at a slower rate (₹40K to ₹70K = 75%)
3. Profit margin improved from ₹10K to ₹30K — a positive trend
4. Slight dip in April revenue and September suggest seasonal

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Perform text visualization by counting word frequency from a paragraph and plotting it.	

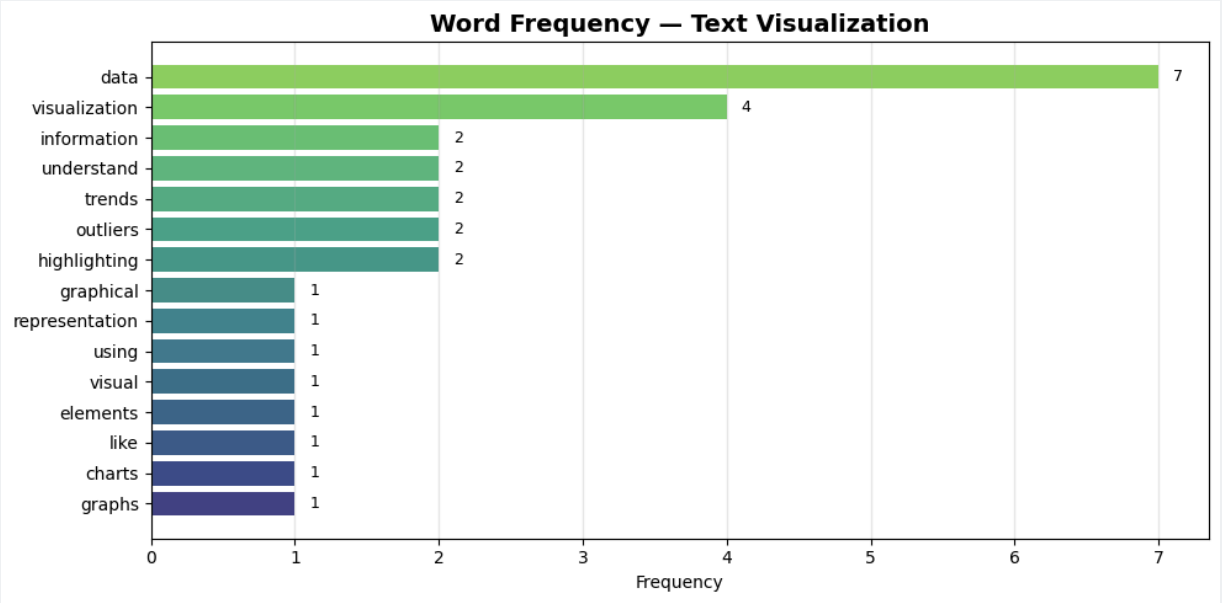
EXPERIMENT NO.: **BCA-609-14**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Perform text visualization by counting word frequency from a paragraph and plotting it.

```
import matplotlib.pyplot as plt
 from collections import Counter
 import re

 text = """
 Data visualization is the graphical representation of data and information.
 By using visual elements like charts, graphs, and maps, data visualization
 tools provide an accessible way to see and understand trends, outliers,
 and patterns in data. Data visualization helps to tell stories by curating
 data into a form easier to understand, highlighting the trends and outliers.
 A good visualization tells a story, removing the noise from data and
 highlighting useful information.
 """

 words = re.findall(r'[a-z]+', text.lower())
 stop_words = {'the','and','a','an','in','to','of','by','is','it','for','from','into','that'}
 filtered = [w for w in words if w not in stop_words and len(w) > 2]

 freq = Counter(filtered)
 top_15 = freq.most_common(15)
 words_list, counts = zip(*top_15)

 plt.figure(figsize=(10, 5))
 bars = plt.barh(list(reversed(words_list)), list(reversed(counts)),
                 color=plt.cm.viridis(np.linspace(0.2, 0.8, 15)))
 plt.xlabel('Frequency')
 plt.title('Word Frequency — Text Visualization', fontsize=14, fontweight='bold')
 plt.grid(axis='x', alpha=0.3)

 import numpy as np
 for bar, count in zip(bars, reversed(counts)):
     plt.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height()/2,
              str(count), va='center', fontsize=9)

 plt.tight_layout()
 plt.show()
```

```
print("\n--- Top 5 Words ---")
for word, count in top_15[:5]:
    print(f"  '{word}' → {count} occurrences")
```

OutPut:



Explanation

Top 5 Words ---
'data' → 7 occurrences
'visualization' → 4 occurrences
'information' → 2 occurrences
'understand' → 2 occurrences
'trends' → 2 occurrences

Prepared By: Ms. Chetna Thakur	Approved By:

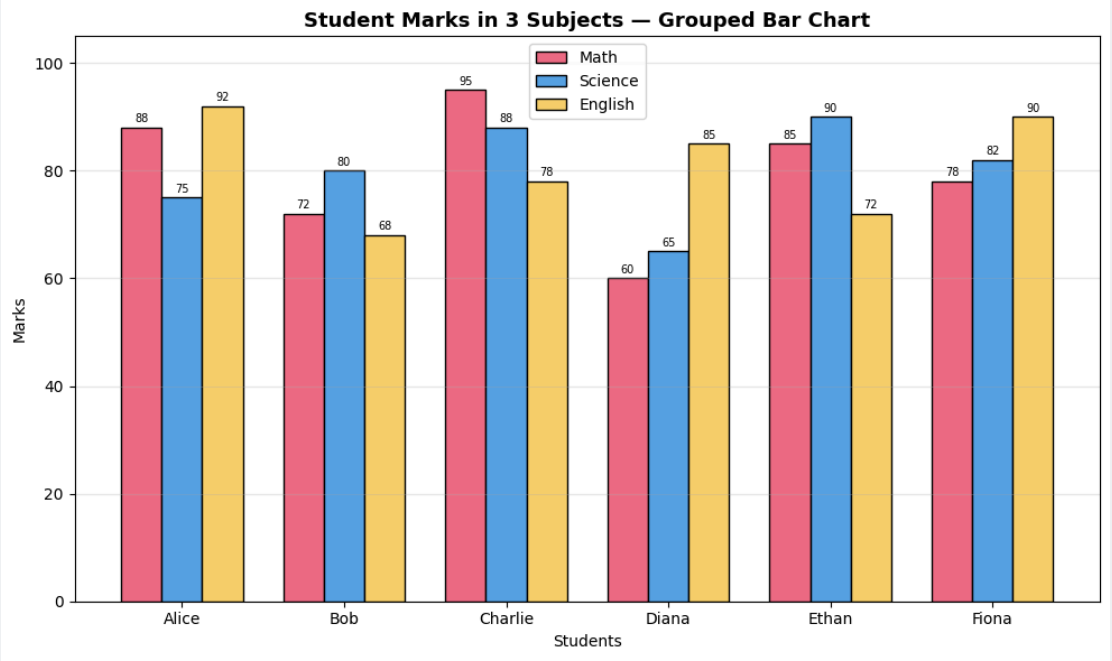
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a simple multivariate chart using more than two variables (e.g., marks in three subjects).	

EXPERIMENT NO.: **BCA-609-15**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create a simple multivariate chart using more than two variables (e.g., marks in three subjects).

```
import matplotlib.pyplot as plt
 import numpy as np

 students = ['Alice', 'Bob', 'Charlie', 'Diana', 'Ethan', 'Fiona']
 math = [88, 72, 95, 60, 85, 78]
 science = [75, 80, 88, 65, 90, 82]
 english = [92, 68, 78, 85, 72, 90]

 x = np.arange(len(students))
 width = 0.25

 fig, ax = plt.subplots(figsize=(10, 6))
 b1 = ax.bar(x - width, math, width, label='Math', color='#FF6384', edgecolor='black')
 b2 = ax.bar(x, science, width, label='Science', color='#36A2EB', edgecolor='black')
 b3 = ax.bar(x + width, english, width, label='English', color='#FFCE56', edgecolor='black')

 ax.set_xlabel('Students')
 ax.set_ylabel('Marks')
 ax.set_title('Student Marks in 3 Subjects — Grouped Bar Chart', fontsize=13, fontweight='bold')
 ax.set_xticks(x)
 ax.set_xticklabels(students)
 ax.legend()
 ax.grid(axis='y', alpha=0.3)
 ax.set_ylim(0, 105)

 for bars in [b1, b2, b3]:
    for bar in bars:
       ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
             str(int(bar.get_height())), ha='center', fontsize=7)

 plt.tight_layout()
 plt.show()

 print("\n--- Explanation ---")
 print("This grouped bar chart compares 3 variables (subjects) for each student.")
 print("Charlie excels in Math (95), Ethan in Science (90), Alice in English (92).")
```

OutPut:



Explanation

This grouped bar chart compares 3 variables (subjects) for each student.
Charlie excels in Math (95), Ethan in Science (90), Alice in English (92).

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Study a small case example where visualization helps in understanding data easily	

EXPERIMENT NO.: **BCA-609-16**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Study a small case example where visualization helps in understanding data easily.

```
import pandas as pd
 import matplotlib.pyplot as plt

 print("=" * 60)
 print("CASE STUDY: COVID-19 Vaccination Drive — District Analysis")
 print("=" * 60)

 data = pd.DataFrame({
    'District': ['Ambala', 'Karnal', 'Panipat', 'Hisar', 'Rohtak'],
    'Population': [150000, 120000, 180000, 200000, 160000],
    'Vaccinated': [135000, 96000, 144000, 140000, 128000],
    'Centers': [12, 8, 15, 10, 11]
 })
 data['Coverage_%'] = (data['Vaccinated'] / data['Population'] * 100).round(1)

 print("\n--- Raw Data ---")
 print(data.to_string(index=False))

 fig, axes = plt.subplots(1, 2, figsize=(14, 5))

 colors = ['#4CAF50' if c >= 80 else '#FF9800' if c >= 70 else '#F44336' for c in data['Coverage_%']]
 bars = axes[0].barh(data['District'], data['Coverage_%'], color=colors, edgecolor='black')
 axes[0].axvline(x=80, color='green', linestyle='--', alpha=0.7, label='Target (80%)')
 axes[0].set_xlabel('Vaccination Coverage (%)')
 axes[0].set_title('Vaccination Coverage by District', fontweight='bold')
 axes[0].legend()
 for bar, pct in zip(bars, data['Coverage_%']):
    axes[0].text(bar.get_width()+0.5, bar.get_y()+bar.get_height()/2, f'{pct}%', va='center')

 axes[1].scatter(data['Centers'], data['Coverage_%'], s=data['Population']/500,
             c=colors, edgecolors='black', alpha=0.8)
 for _, row in data.iterrows():
    axes[1].annotate(row['District'], (row['Centers'], row['Coverage_%']),
                 textcoords="offset points", xytext=(5,5), fontsize=9)
 axes[1].set_xlabel('Number of Vaccination Centers')
 axes[1].set_ylabel('Coverage (%)')
 axes[1].set_title('Centers vs Coverage (Bubble = Population)', fontweight='bold')
 axes[1].grid(True, alpha=0.3)
```

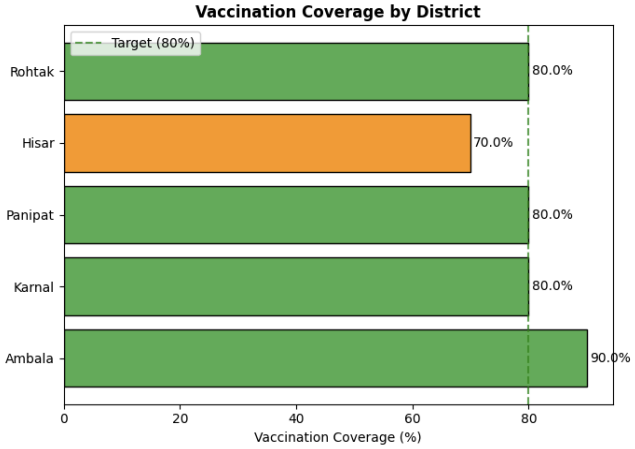
```
plt.tight_layout()
plt.show()

print("\n--- Key Insights from Visualization ---")
print("1. Ambala achieves highest coverage (90%) despite mid-range population")
print("2. Hisar has lowest coverage (70%) despite largest population — needs more centers")
print("3. Visual reveals that Hisar (10 centers for 200K people) is under-resourced")
print("4. Without visualization, these patterns are hard to spot in raw numbers!")
```

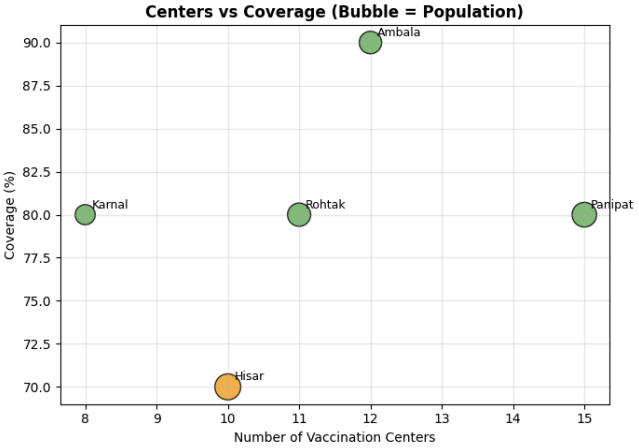
CASE STUDY: COVID-19 Vaccination Drive — District Analysis

Raw Data

District	Population	Vaccinated	Centers	Coverage_%
Ambala	150000	135000	12	90.0
Karnal	120000	96000	8	80.0
Panipat	180000	144000	15	80.0
Hisar	200000	140000	10	70.0
Rohtak	160000	128000	11	80.0

Output:





Explanation

1. Ambala achieves highest coverage (90%) despite mid-range population
2. Hisar has lowest coverage (70%) despite largest population — needs more centers
3. Visual reveals that Hisar (10 centers for 200K people) is under-resourced
4. Without visualization, these patterns are hard to spot in raw numbers!

Prepared By: Ms. Chetna Thakur	Approved By:

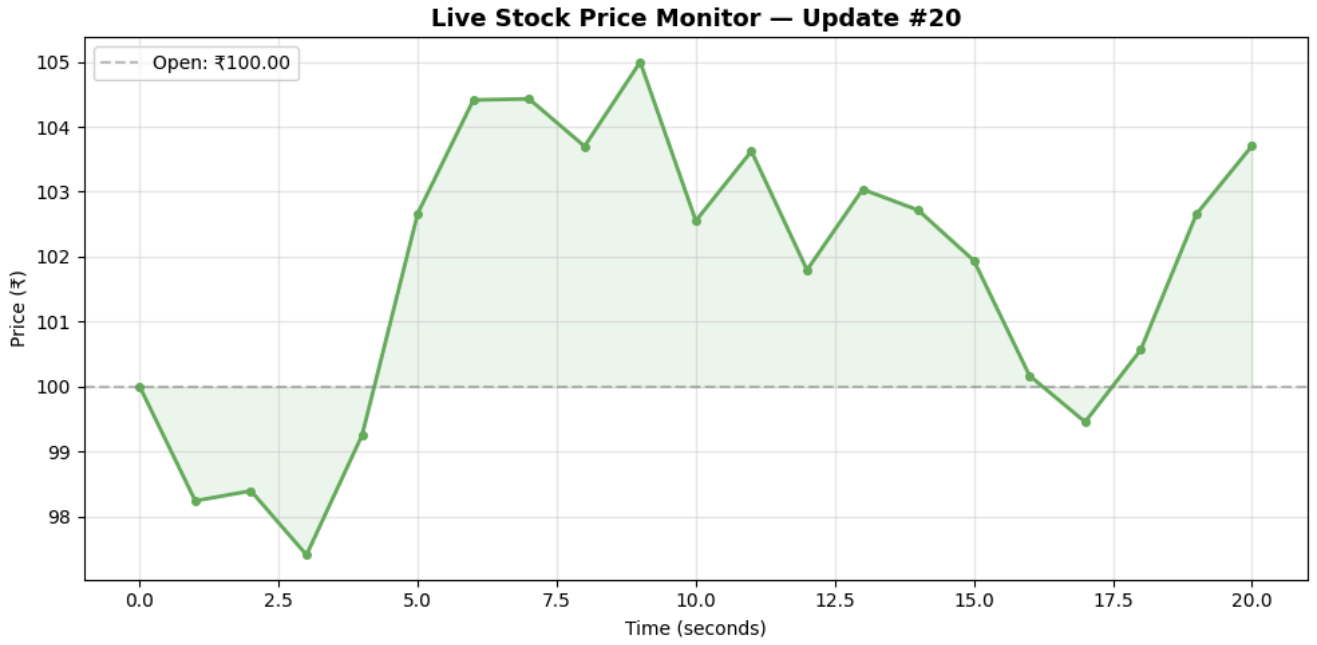
MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a live-like data visualization using continuously updated values.	

EXPERIMENT NO.: **BCA-609-17**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create a live-like data visualization using continuously updated values.

```
import matplotlib.pyplot as plt
 import numpy as np
 from IPython.display import display, clear_output
 import time

 %matplotlib inline

 print("Simulating live stock price updates (20 iterations)...")

 prices = [100]
 timestamps = [0]

 for i in range(1, 21):
    change = np.random.normal(0, 2)
    prices.append(prices[-1] + change)
    timestamps.append(i)

    fig, ax = plt.subplots(figsize=(10, 5))
    color = "#4CAF50" if prices[-1] >= prices[0] else "#F44336"
    ax.plot(timestamps, prices, "-o", color=color, linewidth=2, markersize=4)
    ax.fill_between(timestamps, prices, prices[0], alpha=0.1, color=color)
    ax.axhline(y=prices[0], color="gray", linestyle="--", alpha=0.5, label=f"Open: ₹{prices[0]:.2f}")
    ax.set_title(f"Live Stock Price Monitor — Update #{i}", fontsize=13, fontweight="bold")
    ax.set_xlabel("Time (seconds)")
    ax.set_ylabel("Price (₹)")
    ax.legend(loc="upper left")
    ax.grid(True, alpha=0.3)
    plt.tight_layout()

    clear_output(wait=True)
    display(fig)
    plt.close(fig)
    time.sleep(0.3)

 print(f"Final Price: ₹{prices[-1]:.2f} | Change: {prices[-1]-prices[0]:+.2f}")
 print("Note: This simulates live updates. In production, data comes from APIs/sensors.")
```

Output:



Explanation:

Final Price: ₹103.71 | Change: +3.71

Note: This simulates live updates. In production, data comes from APIs/sensors.

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Write basic rules or best practices for creating good visualizations with examples.	

EXPERIMENT NO.: **BCA-609-18**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Write basic rules or best practices for creating good visualizations with examples.

```
import matplotlib.pyplot as plt
 import numpy as np

 print("=" * 60)
 print("BEST PRACTICES FOR CREATING GOOD DATA VISUALIZATIONS")
 print("=" * 60)

 rules = {
    "1. Choose the Right Chart Type": "Bar for comparison, Line for trends, Pie for proportions",
    "2. Keep It Simple": "Avoid 3D effects, excessive colors, or chart junk",
    "3. Use Clear Labels & Titles": "Always label axes, add meaningful titles",
    "4. Use Consistent Color Scheme": "Use harmonious colors; highlight important data",
    "5. Tell a Story": "Guide the viewer through insights, not just raw data",
    "6. Avoid Misleading Scales": "Start y-axis at 0 (for bar charts); don't truncate",
    "7. Make It Accessible": "Use colorblind-friendly palettes; add annotations",
    "8. Less Is More": "Remove unnecessary gridlines, borders, and decorations",
 }

 for rule, desc in rules.items():
    print(f"\n{rule}")
    print(f"   → {desc}")

 # Visual example: Good vs Bad
 fig, axes = plt.subplots(1, 2, figsize=(14, 5))

 data = [25, 35, 20, 15, 5]
 labels = ['A', 'B', 'C', 'D', 'E']

 # BAD example
 axes[0].pie(data, labels=labels, colors=['red','blue','green','yellow','purple'],
          shadow=True, explode=(0.1,0.1,0.1,0.1,0.1))
 axes[0].set_title(' BAD: Overloaded Pie Chart', fontweight='bold', color='red')

 # GOOD example
 colors_good = ['#FF6384','#36A2EB','#FFCE56','#4BC0C0','#9966FF']
 bars = axes[1].bar(labels, data, color=colors_good, edgecolor='black', linewidth=0.8)
 for bar, val in zip(bars, data):
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
```

```
            str(val), ha='center', fontweight='bold')
axes[1].set_title('GOOD: Clean Bar Chart', fontweight='bold', color='green')
axes[1].set_ylabel('Value')
axes[1].grid(axis='y', alpha=0.3)
axes[1].set_ylim(0, 42)

plt.suptitle('Good vs Bad Visualization Comparison', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

BEST PRACTICES FOR CREATING GOOD DATA VISUALIZATIONS

1. Choose the Right Chart Type
   → Bar for comparison, Line for trends, Pie for proportions

2. Keep It Simple
   → Avoid 3D effects, excessive colors, or chart junk

3. Use Clear Labels & Titles
   → Always label axes, add meaningful titles

4. Use Consistent Color Scheme
   → Use harmonious colors; highlight important data

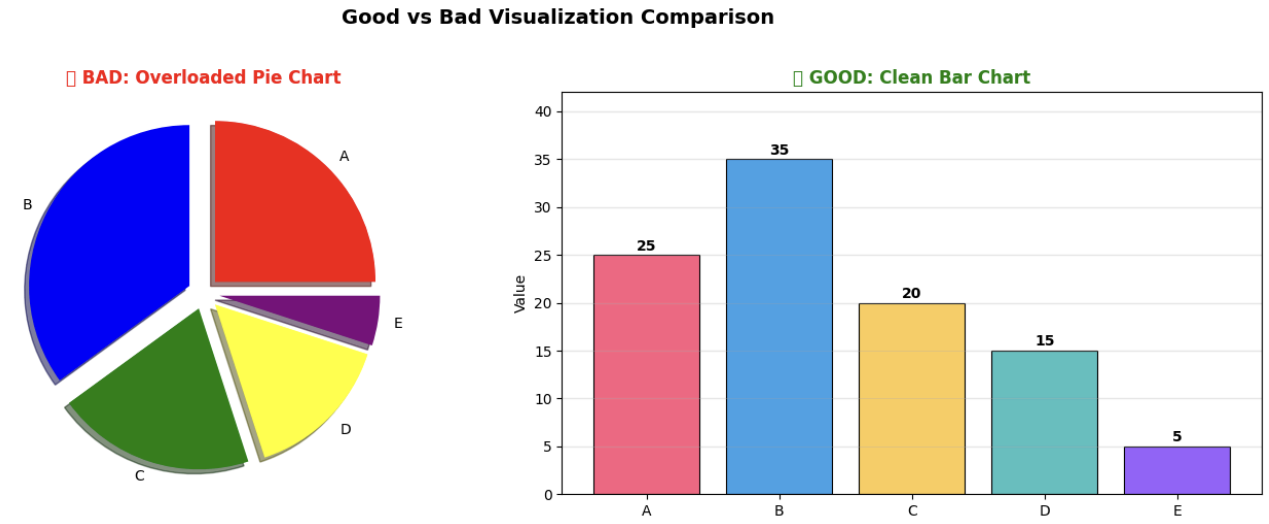
5. Tell a Story
   → Guide the viewer through insights, not just raw data

6. Avoid Misleading Scales
   → Start y-axis at 0 (for bar charts); don't truncate

7. Make It Accessible
   → Use colorblind-friendly palettes; add annotations

8. Less Is More
   → Remove unnecessary gridlines, borders, and decorations

Output:



Prepared By: Ms. Chetna Thakur	Approved By:

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a basic chart to show open and closed ports from given data.	

EXPERIMENT NO.: **BCA-609-19**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

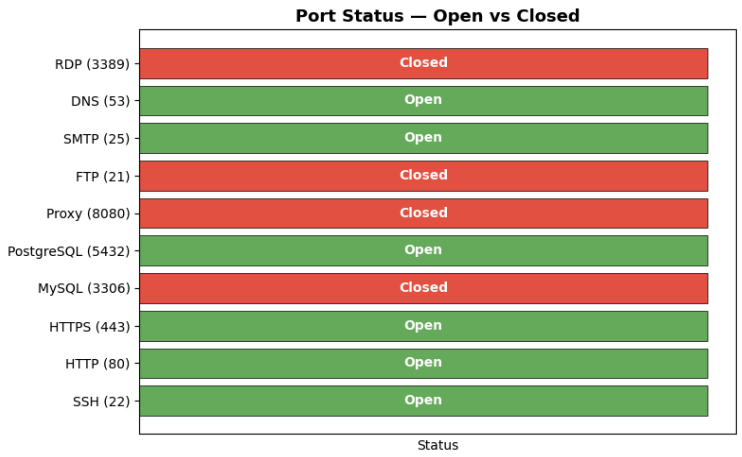
Objective: Create a basic chart to show open and closed ports from given data.

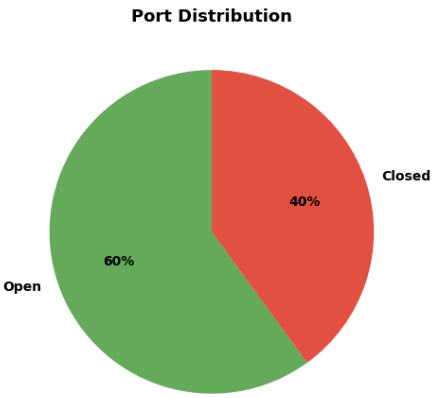
```
import matplotlib.pyplot as plt
 import pandas as pd

 ports_data = pd.DataFrame({
    'Port': [22, 80, 443, 3306, 5432, 8080, 21, 25, 53, 3389],
    'Service': ['SSH', 'HTTP', 'HTTPS', 'MySQL', 'PostgreSQL',
              'Proxy', 'FTP', 'SMTP', 'DNS', 'RDP'],
    'Status': ['Open', 'Open', 'Open', 'Closed', 'Open',
             'Closed', 'Closed', 'Open', 'Open', 'Closed']
 })

 print("--- Port Scan Results ---")
 print(ports_data.to_string(index=False))

 fig, axes = plt.subplots(1, 2, figsize=(14, 5))

 colors = ['#4CAF50' if s == 'Open' else '#F44336' for s in ports_data['Status']]
 bars = axes[0].barh(ports_data['Service'] + ' (' + ports_data['Port'].astype(str) + ')',
                     [1]*len(ports_data), color=colors, edgecolor='black', linewidth=0.5)
 axes[0].set_title('Port Status — Open vs Closed', fontsize=13, fontweight='bold')
 axes[0].set_xlabel('Status')
 axes[0].set_xticks([])
 for bar, status in zip(bars, ports_data['Status']):
    axes[0].text(0.5, bar.get_y()+bar.get_height()/2, status,
                 ha='center', va='center', fontweight='bold', color='white')

 status_counts = ports_data['Status'].value_counts()
 axes[1].pie(status_counts, labels=status_counts.index, autopct='%1.0f%%',
             colors=['#4CAF50', '#F44336'], startangle=90, textprops={'fontweight': 'bold'})
 axes[1].set_title('Port Distribution', fontsize=13, fontweight='bold')

 plt.tight_layout()
 plt.show()

 print(f"\nOpen Ports: {status_counts.get('Open',0)} | Closed Ports: {status_counts.get('Closed',0)}")
 print("Security Note: Unused open ports should be closed to reduce attack surface.")
```

```
--- Port Scan Results ---
 Port    Service Status
```

22 SSH Open
80 HTTP Open
443 HTTPS Open
3306 MySQL Closed
5432 PostgreSQL Open
8080 Proxy Closed
21 FTP Closed
25 SMTP Open
53 DNS Open
3389 RDP Closed

Output:





Explanation:

Open Ports: 6 | Closed Ports: 4
Security Note: Unused open ports should be closed to reduce attack surface.

Prepared By: Ms. Chetna Thakur	Approved By:

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Create a firewall log visualization to identify suspicious traffic patterns.	

EXPERIMENT NO.: **BCA-609-20**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Create a firewall log visualization to identify suspicious traffic patterns.

```
import pandas as pd
 import numpy as np
 import matplotlib.pyplot as plt

 np.random.seed(42)
 n = 200
 hours = np.random.choice(range(24), n, p=[
    0.02,0.01,0.01,0.01,0.02,0.02,0.03,0.04,0.05,0.06,0.06,0.05,
    0.05,0.05,0.06,0.06,0.05,0.04,0.03,0.03,0.04,0.08,0.07,0.06])
 actions = np.random.choice(['ALLOW', 'BLOCK', 'DROP'], n, p=[0.6, 0.25, 0.15])
 src_ips = [f'192.168.1.{np.random.randint(1,50)}' for _ in range(n)]
 protocols = np.random.choice(['TCP', 'UDP', 'ICMP'], n, p=[0.7, 0.2, 0.1])

 logs = pd.DataFrame({'Hour': hours, 'Action': actions, 'Source_IP': src_ips, 'Protocol': protocols})

 print("--- Sample Firewall Logs ---")
 print(logs.head(10).to_string(index=False))

 fig, axes = plt.subplots(2, 2, figsize=(14, 10))

 # Traffic by hour
 hourly = logs.groupby(['Hour', 'Action']).size().unstack(fill_value=0)
 hourly.plot(kind='bar', stacked=True, ax=axes[0,0],
          color={'ALLOW':'#4CAF50','BLOCK':'#FF9800','DROP':'#F44336'})
 axes[0,0].set_title('Traffic by Hour', fontweight='bold')
 axes[0,0].set_xlabel('Hour of Day')
 axes[0,0].legend(title='Action')

 # Action distribution
 logs['Action'].value_counts().plot.pie(ax=axes[0,1], autopct='%1.1f%%',
    colors=['#4CAF50','#FF9800','#F44336'], startangle=90)
 axes[0,1].set_title('Action Distribution', fontweight='bold')
 axes[0,1].set_ylabel('')

 # Top blocked IPs
 blocked = logs[logs['Action'].isin(['BLOCK','DROP'])]
 top_blocked = blocked['Source_IP'].value_counts().head(8)
 top_blocked.plot.barh(ax=axes[1,0], color='#F44336', edgecolor='black')
```

```
axes[1,0].set_title('Top Blocked/Dropped Source IPs', fontweight='bold')
axes[1,0].set_xlabel('Count')

# Protocol breakdown
proto_action = pd.crosstab(logs['Protocol'], logs['Action'])
proto_action.plot(kind='bar', ax=axes[1,1], color=['#4CAF50','#FF9800','#F44336'])
axes[1,1].set_title('Protocol vs Action', fontweight='bold')
axes[1,1].set_xticklabels(axes[1,1].get_xticklabels(), rotation=0)

plt.suptitle('Firewall Log Analysis Dashboard', fontsize=15, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

print("\n--- Suspicious Patterns ---")
print(f"Total blocked/dropped: {len(blocked)} out of {n} ({len(blocked)/n*100:.0f}%)")
sus = top_blocked[top_blocked>5]
if len(sus)>0:
    print(f"Suspicious IPs (>5 blocks): {', '.join(sus.index.tolist())}")
```

Sample Firewall Logs

Hour	Action	Source_IP	Protocol
11	BLOCK	192.168.1.41	TCP
23	ALLOW	192.168.1.30	UDP
19	ALLOW	192.168.1.17	ICMP
15	DROP	192.168.1.49	UDP
7	BLOCK	192.168.1.20	TCP
7	ALLOW	192.168.1.48	TCP
4	ALLOW	192.168.1.25	TCP
21	BLOCK	192.168.1.22	ICMP
16	ALLOW	192.168.1.13	TCP
18	ALLOW	192.168.1.19	TCP

Output:

Explanation:

Peak attack hour: 21:00
Critical alerts: 15
Most common attack: Brute Force (24 incidents)

Prepared By: Chetna Thakur	Approved By: Principal

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Design an intrusion detection log visualization and highlight possible attacks.	

EXPERIMENT NO.: **BCA-609-21**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Design an intrusion detection log visualization and highlight possible attacks.

```
import pandas as pd
 import numpy as np
 import matplotlib.pyplot as plt

 np.random.seed(100)
 n = 150
 attack_types = ['Port Scan', 'Brute Force', 'DDoS', 'SQL Injection', 'XSS', 'Normal']
 severities = ['Critical', 'High', 'Medium', 'Low']

 ids_logs = pd.DataFrame({
    'Timestamp_Hour': np.random.choice(range(24), n),
    'Alert_Type': np.random.choice(attack_types, n, p=[0.15,0.15,0.1,0.1,0.1,0.4]),
    'Severity': np.random.choice(severities, n, p=[0.1,0.2,0.35,0.35]),
    'Source_IP': [f'10.0.0.{np.random.randint(1,30)}' for _ in range(n)],
 })

 attacks_only = ids_logs[ids_logs['Alert_Type'] != 'Normal']
 print(f"Total Alerts: {n} | Attacks Detected: {len(attacks_only)}")
 print("\n--- Sample IDS Alerts ---")
 print(ids_logs.head(10).to_string(index=False))

 fig, axes = plt.subplots(2, 2, figsize=(14, 10))

 # Attack type distribution
 atk_counts = attacks_only['Alert_Type'].value_counts()
 colors_atk = ['#e74c3c','#e67e22','#f1c40f','#9b59b6','#3498db']
 atk_counts.plot.bar(ax=axes[0,0], color=colors_atk, edgecolor='black')
 axes[0,0].set_title('Attack Type Distribution', fontweight='bold')
 axes[0,0].set_xticklabels(axes[0,0].get_xticklabels(), rotation=45, ha='right')

 # Severity breakdown
 sev_colors = {'Critical':'#e74c3c','High':'#e67e22','Medium':'#f1c40f','Low':'#27ae60'}
 sev_counts = attacks_only['Severity'].value_counts()
 sev_counts.plot.pie(ax=axes[0,1], autopct='%1.0f%%',
    colors=[sev_colors.get(s,'gray') for s in sev_counts.index], startangle=90)
 axes[0,1].set_title('Severity Levels', fontweight='bold')
 axes[0,1].set_ylabel('')
```

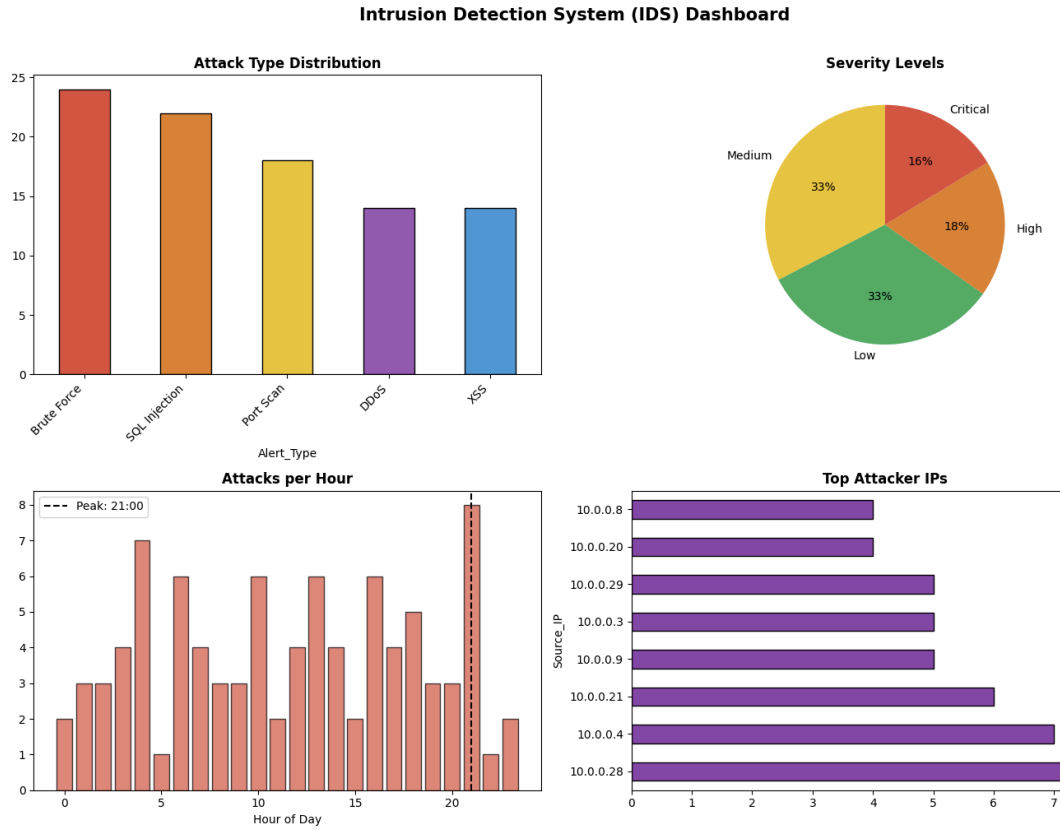
```
# Timeline — attacks per hour
hourly_attacks = attacks_only.groupby('Timestamp_Hour').size()
full_hours = pd.Series(0, index=range(24))
full_hours.update(hourly_attacks)
axes[1,0].bar(full_hours.index, full_hours.values, color='#e74c3c', alpha=0.7, edgecolor='black')
peak = full_hours.idxmax()
axes[1,0].axvline(x=peak, color='black', linestyle='--', label=f'Peak: {peak}:00')
axes[1,0].set_title('Attacks per Hour', fontweight='bold')
axes[1,0].set_xlabel('Hour of Day')
axes[1,0].legend()

# Top suspicious IPs
top_ips = attacks_only['Source_IP'].value_counts().head(8)
top_ips.plot.barh(ax=axes[1,1], color='#8e44ad', edgecolor='black')
axes[1,1].set_title('Top Attacker IPs', fontweight='bold')

plt.suptitle('Intrusion Detection System (IDS) Dashboard', fontsize=15, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

print(f"\n--- Key Findings ---")
print(f"Peak attack hour: {peak}:00")
crit = len(attacks_only[attacks_only['Severity']=='Critical'])
print(f"Critical alerts: {crit}")
print(f"Most common attack: {atk_counts.index[0]} ({atk_counts.iloc[0]} incidents)")
```

```
Total Alerts: 150 | Attacks Detected: 92
```

Sample IDS Alerts

Timestamp_Hour	Alert_Type	Severity	Source_IP
8	Brute Force	Medium	10.0.0.16
3	Normal	Low	10.0.0.19
7	Normal	Medium	10.0.0.13
23	Normal	Medium	10.0.0.22
15	Normal	Low	10.0.0.24
16	DDoS	Low	10.0.0.4
10	Normal	Low	10.0.0.16
20	Normal	High	10.0.0.29
2	Normal	Low	10.0.0.16
21	Normal	Medium	10.0.0.10

Output:



Explanation:

Open Ports: 6 | Closed Ports: 4
Security Note: Unused open ports should be closed to reduce attack surface.

Prepared By: Ms. Chetna Thakur	Approved By:

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Write a program to implement sentiment analysis on tweet-like data using Python libraries.	

EXPERIMENT NO.: **BCA-609-22**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Write a program to implement sentiment analysis on tweet-like data using Python libraries.

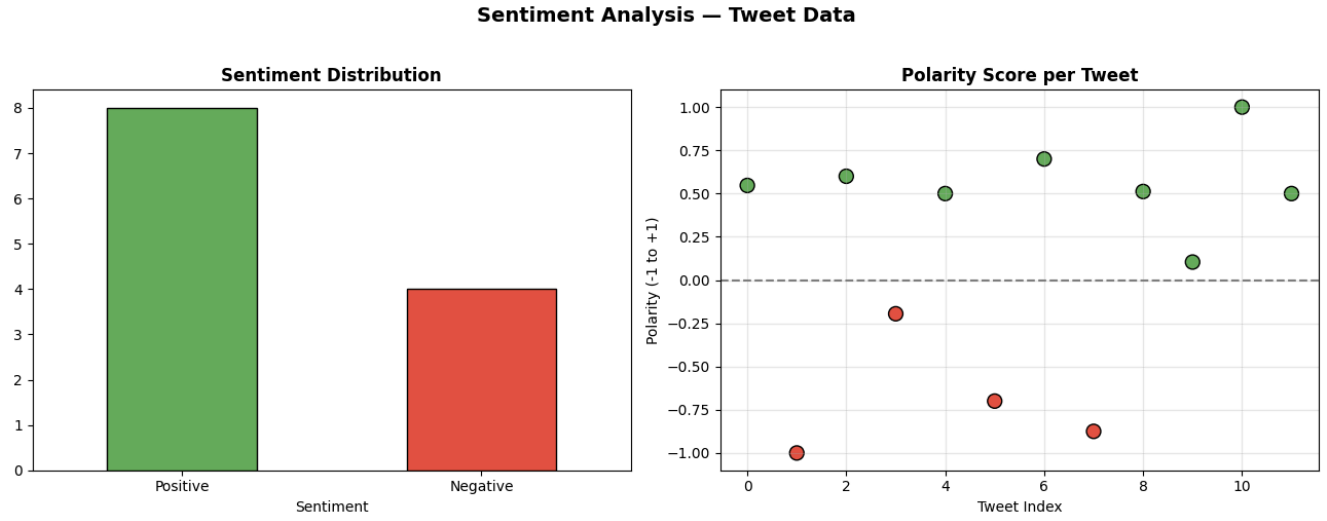
```
try:
    from textblob import TextBlob
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'textblob', '-q'])
    from textblob import TextBlob
import matplotlib.pyplot as plt
import pandas as pd

# Simulated tweet data (since Twitter API requires authentication)
tweets = [
    "I love this new phone, it's absolutely amazing! #happy",
    "Terrible customer service, never buying again",
    "The weather is nice today",
    "This product is a total waste of money!!!",
    "Great experience at the restaurant, highly recommend!",
    "I'm so frustrated with the delivery delays",
    "Just finished a good book, feeling content",
    "Worst experience ever, completely disappointed",
    "The new update is fantastic, great job developers!",
    "Average movie, nothing special about it",
    "Absolutely thrilled with my purchase!!! Best decision ever",
    "The service was okay, could be better though",
]

results = []
for tweet in tweets:
    blob = TextBlob(tweet)
    polarity = blob.sentiment.polarity
    sentiment = 'Positive' if polarity > 0.1 else 'Negative' if polarity < -0.1 else 'Neutral'
    results.append({'Tweet': tweet[:50]+'...' if len(tweet)>50 else tweet,
                    'Polarity': round(polarity, 3), 'Sentiment': sentiment})

df = pd.DataFrame(results)
print("--- Sentiment Analysis Results ---")
print(df.to_string(index=False))
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Sentiment distribution
counts = df['Sentiment'].value_counts()
colors = {'Positive':'#4CAF50','Negative':'#F44336','Neutral':'#FF9800'}
counts.plot.bar(ax=axes[0], color=[colors[s] for s in counts.index], edgecolor='black')
axes[0].set_title('Sentiment Distribution', fontweight='bold')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=0)

# Polarity scatter
c = [colors[s] for s in df['Sentiment']]
axes[1].scatter(range(len(df)), df['Polarity'], c=c, s=100, edgecolors='black')
axes[1].axhline(y=0, color='gray', linestyle='--')
axes[1].set_title('Polarity Score per Tweet', fontweight='bold')
axes[1].set_xlabel('Tweet Index')
axes[1].set_ylabel('Polarity (-1 to +1)')
axes[1].grid(True, alpha=0.3)

plt.suptitle('Sentiment Analysis — Tweet Data', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

pos = len(df[df['Sentiment']=='Positive'])
neg = len(df[df['Sentiment']=='Negative'])
print(f"\nPositive: {pos} | Negative: {neg} | Neutral: {len(df)-pos-neg}")
print(f"Overall Sentiment: {'Positive ' if pos>neg else 'Negative ' if neg>pos else 'Mixed '}")
```

Sentiment Analysis Results

Tweet	Polarity	Sentiment
I love this new phone, it's absolutely amazing! #h...	0.547	Positive
Terrible customer service, never buying again	-1.000	Negative
The weather is nice today	0.600	Positive
This product is a total waste of money!!!	-0.195	Negative
Great experience at the restaurant, highly recomme...	0.500	Positive
I'm so frustrated with the delivery delays	-0.700	Negative
Just finished a good book, feeling content	0.700	Positive
Worst experience ever, completely disappointed	-0.875	Negative
The new update is fantastic, great job developers!	0.512	Positive
Average movie, nothing special about it	0.104	Positive
Absolutely thrilled with my purchase!!! Best decis...	1.000	Positive
The service was okay, could be better though	0.500	Positive

Output:



Explanation:

Positive: 8 | Negative: 4 | Neutral: 0
Overall Sentiment: Positive

Prepared By: Ms. Chetna Thakur	Approved By:

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Write a program to create interactive visualizations to monitor device activity for pre-processed simulated IoT data streams in real-time	

EXPERIMENT NO.: **BCA-609-23**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

Objective: Write a program to create interactive visualizations to monitor device activity for pre-processed simulated IoT data streams in real-time.

```
try:
    import plotly.express as px
    import plotly.graph_objects as go
    from plotly.subplots import make_subplots
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'plotly', '-q'])
    import plotly.express as px
    import plotly.graph_objects as go
    from plotly.subplots import make_subplots
import pandas as pd
import numpy as np

np.random.seed(42)

devices = ['Thermostat', 'Camera', 'Door_Sensor', 'Light', 'Smoke_Detector']
hours = list(range(24))
records = []
for device in devices:
    for h in hours:
        base_temp = 25 + np.random.normal(0, 3)
        records.append({
            'Device': device, 'Hour': h,
            'Temperature': round(base_temp, 1),
            'Status': np.random.choice(['Active','Idle','Alert'], p=[0.7,0.2,0.1]),
            'Battery': max(10, min(100, int(80 + np.random.normal(0, 15)))),
            'Data_Packets': np.random.randint(10, 200)
        })

iot_data = pd.DataFrame(records)

print("--- IoT Device Data (Sample) ---")
print(iot_data.head(10).to_string(index=False))

fig = make_subplots(rows=2, cols=2,
    subplot_titles=('Device Activity Timeline', 'Battery Levels',
```

```
                'Data Packets by Device', 'Status Distribution'))

for device in devices:
    dev_data = iot_data[iot_data['Device'] == device]
    fig.add_trace(go.Scatter(x=dev_data['Hour'], y=dev_data['Data_Packets'],
                             name=device, mode='lines+markers'), row=1, col=1)

avg_battery = iot_data.groupby('Device')['Battery'].mean()
fig.add_trace(go.Bar(x=avg_battery.index, y=avg_battery.values,
                     marker_color=['#4CAF50','#2196F3','#FF9800','#9C27B0','#F44336']),
              row=1, col=2)

fig.add_trace(go.Box(x=iot_data['Device'], y=iot_data['Data_Packets']), row=2, col=1)

status_counts = iot_data.groupby(['Device','Status']).size().unstack(fill_value=0)
for status in ['Active','Idle','Alert']:
    if status in status_counts.columns:
        fig.add_trace(go.Bar(name=status, x=status_counts.index,
                             y=status_counts[status]), row=2, col=2)

fig.update_layout(height=700, title_text='IoT Device Monitoring Dashboard',
                  title_font_size=16, showlegend=True)
fig.show()

alerts = iot_data[iot_data['Status']=='Alert']
print(f"\nTotal Records: {len(iot_data)} | Alerts: {len(alerts)}")
print(f"Alert Rate: {len(alerts)/len(iot_data)*100:.1f}%")
for d in devices:
    dev_alerts = len(alerts[alerts['Device']==d])
    if dev_alerts > 0:
        print(f"  {d}: {dev_alerts} alerts")
```

IoT Device Data (Sample)

	Device	Hour	Temperature	Status	Battery	Data_Packets
Thermostat	0	26.5	Idle	77	198	
Thermostat	1	21.4	Active	100	97	
Thermostat	2	22.3	Active	73	62	
Thermostat	3	23.3	Active	72	67	
Thermostat	4	17.2	Active	94	68	
Thermostat	5	23.4	Alert	78	24	
Thermostat	6	29.4	Active	76	64	
Thermostat	7	24.9	Idle	85	144	
Thermostat	8	23.2	Active	94	93	
Thermostat	9	27.3	Active	63	17	

Unable to display output for mime type(s): application/vnd.plotly.v1+json

Explanation:

Total Records: 120 | Alerts: 12
Alert Rate: 10.0%
  Thermostat: 2 alerts
  Door_Sensor: 4 alerts
  Light: 2 alerts
  Smoke_Detector: 4 alerts

Prepared By: Ms. Chetna Thakur	**Approved By:**

MMICT&BM	MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA	LABORATORY MANUAL
	PRACTICAL EXPERIMENT INSTRUCTION SHEET	
	Design a secured data visualization system architecture and explain the security measures used.	

EXPERIMENT NO.: **BCA-609-24**	ISSUE No. : 01	ISSUE DATE:
REV. DATE:	REV. No.: 00	Department: MCA
LABORATORY: Data Visualization Lab	Semester-VI	Page:

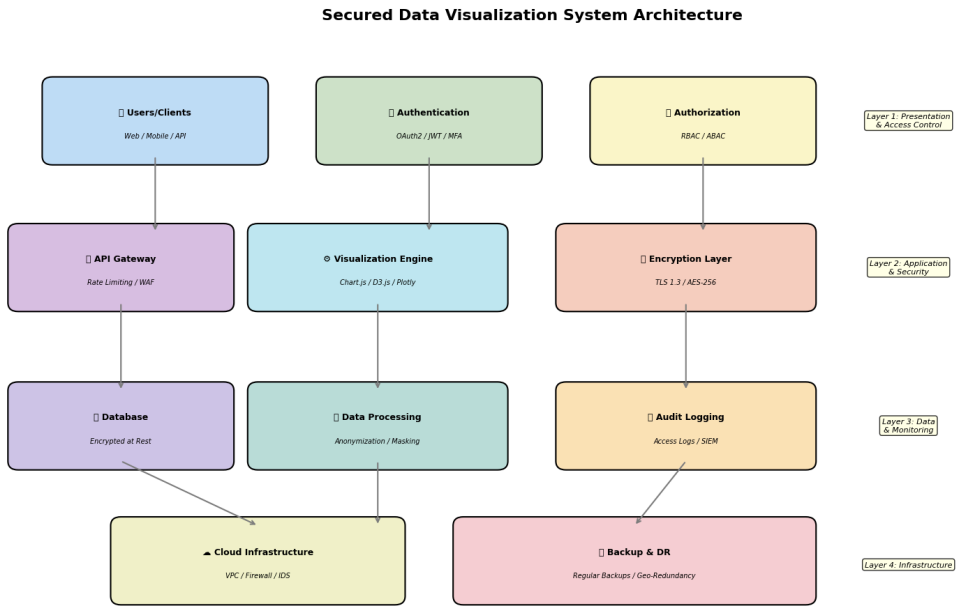
Objective: Design a secured data visualization system architecture and explain the security measures used.

```
import matplotlib.pyplot as plt
 import matplotlib.patches as mpatches

 fig, ax = plt.subplots(figsize=(16, 10))
 ax.set_xlim(0, 16)
 ax.set_ylim(0, 11)
 ax.axis('off')
 ax.set_title('Secured Data Visualization System Architecture', fontsize=16, fontweight='bold', pad=20)

 def draw_box(ax, x, y, w, h, label, color, sublabel=''):
    rect = mpatches.FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.15",
                                   facecolor=color, edgecolor='black', linewidth=1.5)
    ax.add_patch(rect)
    ax.text(x+w/2, y+h/2+0.15, label, ha='center', va='center', fontweight='bold', fontsize=9)
    if sublabel:
        ax.text(x+w/2, y+h/2-0.25, sublabel, ha='center', va='center', fontsize=7, style='italic')

 # Layer 1: Users
 draw_box(ax, 1, 9, 3, 1.2, ' Users/Clients', '#BBDEFB', 'Web / Mobile / API')
 draw_box(ax, 5, 9, 3, 1.2, ' Authentication', '#C8E6C9', 'OAuth2 / JWT / MFA')
 draw_box(ax, 9, 9, 3, 1.2, ' Authorization', '#FFF9C4', 'RBAC / ABAC')

 # Layer 2: Application
 draw_box(ax, 0.5, 6.5, 3, 1.2, ' API Gateway', '#E1BEE7', 'Rate Limiting / WAF')
 draw_box(ax, 4, 6.5, 3.5, 1.2, ' Visualization Engine', '#B2EBF2', 'Chart.js / D3.js / Plotly')
 draw_box(ax, 8.5, 6.5, 3.5, 1.2, ' Encryption Layer', '#FFCCBC', 'TLS 1.3 / AES-256')

 # Layer 3: Data
 draw_box(ax, 0.5, 3.8, 3, 1.2, ' Database', '#D1C4E9', 'Encrypted at Rest')
 draw_box(ax, 4, 3.8, 3.5, 1.2, ' Data Processing', '#B2DFDB', 'Anonymization / Masking')
 draw_box(ax, 8.5, 3.8, 3.5, 1.2, ' Audit Logging', '#FFE0B2', 'Access Logs / SIEM')

 # Layer 4: Infrastructure
 draw_box(ax, 2, 1.5, 4, 1.2, ' Cloud Infrastructure', '#F0F4C3', 'VPC / Firewall / IDS')
 draw_box(ax, 7, 1.5, 5, 1.2, ' Backup & DR', '#FFCDD2', 'Regular Backups / Geo-Redundancy')
```

```
# Arrows
for x1, y1, x2, y2 in [(2.5,9,2.5,7.7), (6.5,9,6.5,7.7), (10.5,9,10.5,7.7),
                       (2,6.5,2,5), (5.75,6.5,5.75,5), (10.25,6.5,10.25,5),
                       (2,3.8,4,2.7), (5.75,3.8,5.75,2.7), (10.25,3.8,9.5,2.7)]:
    ax.annotate('', xy=(x2, y2), xytext=(x1, y1),
                arrowprops=dict(arrowstyle='->', color='gray', lw=1.5))

# Labels
labels_right = [
    (13.5, 9.5, 'Layer 1: Presentation\n& Access Control'),
    (13.5, 7.0, 'Layer 2: Application\n& Security'),
    (13.5, 4.3, 'Layer 3: Data\n& Monitoring'),
    (13.5, 2.0, 'Layer 4: Infrastructure'),
]
for x, y, label in labels_right:
    ax.text(x, y, label, fontsize=8, ha='center', style='italic',
            bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8))

plt.tight_layout()
plt.show()

print("\n" + "=" * 65)
print("SECURITY MEASURES IN THE DATA VISUALIZATION SYSTEM")
print("=" * 65)

measures = [
    ("1. Authentication & Authorization",
     "OAuth2/JWT tokens with Multi-Factor Authentication (MFA)\n"
     "   Role-Based Access Control (RBAC) to restrict data access"),
    ("2. Data Encryption",
     "TLS 1.3 for data in transit; AES-256 for data at rest\n"
     "   End-to-end encryption for sensitive visualizations"),
    ("3. API Security",
     "API Gateway with rate limiting to prevent DDoS\n"
     "   Web Application Firewall (WAF) for input validation"),
    ("4. Data Protection",
     "Data masking and anonymization for PII\n"
     "   Differential privacy for aggregate visualizations"),
    ("5. Audit & Monitoring",
     "Comprehensive audit logging of all data access\n"
     "   SIEM integration for real-time threat detection"),
    ("6. Infrastructure Security",
     "Virtual Private Cloud (VPC) with network segmentation\n"
     "   Intrusion Detection Systems (IDS/IPS)"),
    ("7. Backup & Disaster Recovery",
     "Regular automated backups with encryption\n"
     "   Geo-redundant storage for business continuity"),
```

```
]

for title, desc in measures:
    print(f"\n{title}")
    print(f"   {desc}")
```

Output:



Explanation:

SECURITY MEASURES IN THE DATA VISUALIZATION SYSTEM

1. Authentication & Authorization
   OAuth2/JWT tokens with Multi-Factor Authentication (MFA)
   Role-Based Access Control (RBAC) to restrict data access

2. Data Encryption
   TLS 1.3 for data in transit; AES-256 for data at rest
   End-to-end encryption for sensitive visualizations

3. API Security
   API Gateway with rate limiting to prevent DDoS
   Web Application Firewall (WAF) for input validation

4. Data Protection
   Data masking and anonymization for PII
   Differential privacy for aggregate visualizations

5. Audit & Monitoring
Comprehensive audit logging of all data access
SIEM integration for real-time threat detection

6. Infrastructure Security
Virtual Private Cloud (VPC) with network segmentation
Intrusion Detection Systems (IDS/IPS)

7. Backup & Disaster Recovery
Regular automated backups with encryption
Geo-redundant storage for business continuity

Prepared By: Ms. Chetna Thakur	**Approved By:**